



Universidad Tecnológica de Ciudad Juárez



QualityOps AI: validación preventiva de la calidad de software con DevOps e IA

Estándares y Métricas para el Desarrollo de Software

Prof. Francisco Javier Elizondo Luna

Ivan Valle Cortes – Matrícula: **24110751**

8 de octubre de 2026 -- Grupo DSM51



Educación
Secretaría de Educación Pública



MÁS CHIHUAHUA
más de lo bueno
GOBIERNO DEL ESTADO

UTP
DIRECCIÓN GENERAL DE UNIVERSIDADES
TECNOLÓGICAS Y POLITÉCNICAS

UTCJ

I. INTRODUCCIÓN

Este reporte documenta **QualityOps AI**, el programa que desarrollé para la materia Estándares y Métricas para el Desarrollo de Software. Es una herramienta **DevOps con IA** que, cada vez que alguien propone un cambio en un repositorio de GitHub, ejecuta las pruebas, calcula métricas de producto, de proceso y de proyecto, aplica un *quality gate* (una barrera que impide integrar código que no cumple los umbrales) y publica un informe de calidad con interpretación asistida por un modelo de lenguaje. La herramienta se analiza a sí misma (*dogfooding*): cada cifra de este documento proviene de su propia ejecución sobre su propio repositorio.

El resultado es un programa de 1.066 KLOC con 56 pruebas automatizadas, una cobertura de 88.73 %, un pipeline de GitHub Actions que corre en cada Pull Request y un historial de Git del que se obtienen, sin registro manual, los defectos reales, su tiempo de detección y su tiempo de reparación.

El documento sigue el orden de la actividad. Primero describo el proyecto, por qué es un proyecto DevOps, cómo trabajé y cómo está construido (capítulos 1 a 3). Después respondo los ocho puntos de calidad solicitados (capítulos 4 a 11), presento las métricas calculadas con IA (capítulo 12) y las cuatro técnicas de estimación (capítulo 13). El capítulo 14 es una guía para poner a prueba el programa y reconocer cada salida posible, y el capítulo 15 resume los resultados y sus límites. Cada capítulo comienza con un recuadro **En resumen** que da la respuesta corta; el resto del capítulo la desarrolla y la respalda con evidencia.

Regla de evidencia. Ninguna cifra, tabla, gráfica o captura de este reporte es inventada ni ilustrativa. Las capturas son de mi terminal o de GitHub; las gráficas se generan con `docs/reporte/graficas.py` a partir de `reports/metrics.json`, y los diagramas con Graphviz desde archivos `.dot` versionados en `docs/diagramas/`. Cualquier persona puede repetir cada resultado con los comandos de la sección 3.3.

I. INTRODUCCIÓN	I
1. DESCRIPCIÓN DEL PROYECTO Y JUSTIFICACIÓN DEVOPS.....	1 -
1.1 <i>Qué es QualityOps AI.....</i>	<i>- 1 -</i>
1.2 <i>Problema que resuelve y valor agregado</i>	<i>- 2 -</i>
1.3 <i>Justificación como proyecto DevOps.....</i>	<i>- 3 -</i>
1.4 <i>Tecnologías y frameworks.....</i>	<i>- 3 -</i>
2. METODOLOGÍA DE TRABAJO	- 5 -
2.1 <i>Entorno de trabajo.....</i>	<i>- 5 -</i>
2.2 <i>Kanban y sprints.....</i>	<i>- 6 -</i>
2.3 <i>Convenciones de trabajo</i>	<i>- 7 -</i>
2.4 <i>Fuentes de datos y regla de evidencia</i>	<i>- 8 -</i>
2.5 <i>Uso de IA durante el desarrollo.....</i>	<i>- 8 -</i>
3. ARQUITECTURA, EJECUCIÓN Y REPRODUCCIÓN	- 10 -
3.1 <i>Arquitectura de componentes.....</i>	<i>- 10 -</i>
3.2 <i>Pipeline de integración continua.....</i>	<i>- 12 -</i>
3.3 <i>Cómo reproducir los resultados</i>	<i>- 13 -</i>
3.4 <i>Guía de comandos.....</i>	<i>- 14 -</i>
3.5 <i>Ejecución y salida real</i>	<i>- 14 -</i>
4. CALIDAD DE SOFTWARE Y GRADO DE CUMPLIMIENTO.....	- 16 -
4.1 <i>Definición de calidad de software</i>	<i>- 16 -</i>
4.2 <i>Cómo se mide el grado de cumplimiento</i>	<i>- 16 -</i>
4.3 <i>Resultado del grado de cumplimiento</i>	<i>- 17 -</i>
4.4 <i>Lectura del resultado</i>	<i>- 19 -</i>
5. INFORME DE CALIDAD EN UN FORMATO.....	- 20 -
5.1 <i>Estructura del informe.....</i>	<i>- 20 -</i>
5.2 <i>Reglas del dictamen de liberación.....</i>	<i>- 20 -</i>
5.3 <i>Informe real generado en el pipeline</i>	<i>- 21 -</i>
6. REQUISITOS EXPLÍCITOS	- 24 -
6.1 <i>Catálogo de requisitos funcionales</i>	<i>- 24 -</i>
6.2 <i>Catálogo de requisitos no funcionales</i>	<i>- 24 -</i>
6.3 <i>Especificación por módulo</i>	<i>- 25 -</i>
6.4 <i>Trazabilidad entre requisitos, pruebas y evidencia</i>	<i>- 38 -</i>
7. REQUISITOS DEL USUARIO Y EXPECTATIVAS DEL NEGOCIO	- 39 -
7.1 <i>Usuarios y necesidades</i>	<i>- 39 -</i>
7.2 <i>Historias de usuario</i>	<i>- 39 -</i>
7.3 <i>Expectativas del negocio e indicadores.....</i>	<i>- 40 -</i>
7.4 <i>Requisitos implícitos.....</i>	<i>- 40 -</i>

8. ASEGURAMIENTO DE LA CALIDAD DE SOFTWARE - 41 -

8.1	Plan SQA según IEEE 730	- 41 -
8.2	Gestión de configuración	- 41 -
8.3	Revisiones técnicas	- 42 -
8.4	Verificación y validación	- 42 -
8.5	Defectos registrados	- 43 -

9. CARACTERÍSTICAS DE CALIDAD: IDONEIDAD FUNCIONAL Y FIABILIDAD- 45 -

9.1	Modelo de calidad aplicado	- 45 -
9.2	Idoneidad funcional	- 45 -
9.3	Fiabilidad	- 46 -

10. ENFOQUE PREVENTIVO..... - 48 -

10.1	Concepto	- 48 -
10.2	Mecanismos preventivos implementados	- 48 -
10.3	Demostración de bloqueo	- 48 -
10.4	Detección temprana de los defectos reales	- 51 -

11. CARÁCTER SISTEMÁTICO..... - 53 -

11.1	Ciclo de trabajo repetible	- 53 -
11.2	Trazabilidad de extremo a extremo	- 53 -
11.3	Normas aplicadas	- 53 -
11.4	Mapeo con los procesos de ISO/IEC/IEEE 12207	- 54 -
11.5	Registro de decisiones	- 54 -

12. MÉTRICAS CALCULADAS APLICANDO IA - 56 -

12.1	Cómo participa la IA en las métricas	- 56 -
12.2	Métricas de producto	- 57 -
12.3	Métricas de proceso	- 59 -
12.4	Métricas de proyecto	- 60 -
12.5	Interpretación del AI Advisor	- 62 -

13. TÉCNICAS DE ESTIMACIÓN..... - 63 -

13.1	Línea base	- 63 -
13.2	Juicio de expertos	- 63 -
13.3	Estimación de tres puntos (PERT)	- 64 -
13.4	Puntos de función	- 64 -
13.5	Estimación análoga	- 64 -
13.6	Comparación con las horas reales	- 65 -

14. PRUEBAS DE FUNCIONAMIENTO Y SALIDAS POSIBLES - 66 -

14.1	Escenarios y salidas esperadas	- 66 -
14.2	Guion de demostración	- 66 -

15. RESUMEN DE RESULTADOS Y AMENAZAS A LA VALIDEZ..... - 68 -

15.1	Resumen de resultados	- 68 -
15.2	Amenazas a la validez.....	- 68 -

III. CONCLUSIÓN.....	VII
-----------------------------	------------

IV. REFERENCIAS	VIII
------------------------------	-------------

ÍNDICE DE FIGURAS

Figura 1. Repositorio público en GitHub al cierre del Sprint 0 (evidencia E01_repo_github_1)	2
Figura 2. Creación del entorno: versiones de Python y Git, git init y entorno virtual (evidencia E00_entorno)	5
Figura 3. Dependencias instaladas y versiones resultantes (evidencia E00b_pip_install_2)	6
Figura 4. Tablero Kanban al cierre del Sprint 0 (evidencia E02c_kanban)	7
Figura 5. Commit de corrección con sus trailers en git log (evidencia E15c)	8
Figura 6. Arquitectura de componentes de QualityOps AI	11
Figura 7. Pipeline CI de QualityOps AI y regla de protección de main	12
Figura 8. Log del job «calidad» en GitHub Actions, paso a paso (evidencia E08d)	13
Figura 9. Ejecución final: CLI, quality gate e informe con LLM (evidencia E16a)	15
Figura 10. Estado de los requisitos de la matriz viva por categoría	17
Figura 11. Grado de cumplimiento calculado por el dashboard (evidencia E09b)	18
Figura 12. Informe de calidad publicado en el resumen de GitHub Actions (PR #40) (evidencia E10k) ..	21
Figura 13. Secciones 4 a 6 del informe en Actions (PR #38) (evidencia E10_actions_PR38_summary_3)	22
Figura 14. Sección 10 del informe: interpretación por reglas y del modelo de lenguaje (evidencia E10i) ..	22
Figura 15. Complejidad por función calculada por QualityOps comparada con radon (evidencia E07) ...	26
Figura 16. Conteo de líneas de radon usado para validar el SLOC (evidencia E11c)	27
Figura 17. Cobertura de QualityOps comparada con coverage report (evidencia E06)	28
Figura 18. Minería de defectos sobre main: tres defectos reales (evidencia E11f)	29
Figura 19. Densidad de defectos por archivo (T07, antes del primer defecto) (evidencia E11b)	29
Figura 20. Primera ejecución con métricas de proceso y proyecto (T13–T14) (evidencia E11d)	30
Figura 21. CLI con las cuatro técnicas de estimación (evidencia E12)	31
Figura 22. Primera ejecución de la CLI y estructura de metrics.json (T08) (evidencia E04b)	32
Figura 23. Quality gate bloqueando con un umbral de demostración y aprobando con el real (evidencia E08a_gate)	33
Figura 24. Primera ejecución del pipeline: job «calidad» y resumen del quality gate (PR #35) (evidencia E08_actions)	34
Figura 25. Ruleset proteger-main activo (evidencia E08b)	34
Figura 26. Informe en Actions después de corregir el defecto 1 (evidencia E15b)	35
Figura 27. Tolerancia a fallos: el LLM no responde y el informe se genera por reglas (evidencia E10m) ..	35
Figura 28. Dashboard: pestaña de resumen con el quality gate (evidencia E09a)	36
Figura 29. Dashboard: pestaña de producto (evidencia E09c)	37
Figura 30. Dashboard: pestaña AI Advisor (evidencia E09h)	37
Figura 31. Defecto 1 detectado en la revisión del PR #38 (evidencia E15)	42
Figura 32. Cobertura de líneas por archivo frente al umbral del gate	43
Figura 33. Defecto 4: pytest tardaba 105 segundos (evidencia E10j)	44
Figura 34. Resultado de las ejecuciones del pipeline por tipo de evento	46
Figura 35. Historial de ejecuciones del pipeline (gh run list) (evidencia E16b)	47
Figura 36. Corrección del defecto 4 y modelo inexistente reportado con su detalle (evidencia E10l)	47
Figura 37. El ruleset impide el merge mientras el check obligatorio no termina (PR #41) (evidencia E08g)	49
Figura 38. Check «calidad» en rojo para el PR de demostración (evidencia E17a)	49
Figura 39. Resumen del run bloqueado: complejidad 11 > 10 (evidencia E17b)	50
Figura 40. GitHub impide el merge del PR bloqueado (evidencia E17c)	50
Figura 41. Informe del PR bloqueado: dictamen NO LIBERAR (evidencia E17d)	51
Figura 42. Tiempo de detección y de reparación de cada defecto real	51
Figura 43. Distribución de la complejidad ciclomática por función	58
Figura 44. Densidad de defectos por archivo afectado y global	59
Figura 45. Ciclo de vida de un defecto y dónde se mide cada métrica de proceso	59
Figura 46. Horas estimadas y reales por módulo	61
Figura 47. Estimación por técnica frente a las horas reales registradas	65

ÍNDICE DE TABLAS

Tabla 1. Ficha técnica del proyecto	1
Tabla 2. Prácticas DevOps y cómo se implementan en QualityOps AI	3
Tabla 3. Tecnologías utilizadas	3
Tabla 4. Sprints planeados (pyproject.toml) y tareas asignadas (data/time_log.csv)	6
Tabla 5. Commits del proyecto por tipo (git log, sin merges)	7
Tabla 6. Fuentes de datos del proyecto	8
Tabla 7. Bitácora de uso de IA (docs/AI_LOG.md, resumida)	9
Tabla 8. Módulos del producto con sus métricas (commit 9d93529)	12
Tabla 9. Comandos del programa	14
Tabla 10. Matriz de cumplimiento completa (docs/MATRIZ_CUMPLIMIENTO.md)	18
Tabla 11. Las 12 secciones del informe de calidad	20
Tabla 12. Criterios del dictamen	20
Tabla 13. Requisitos funcionales	24
Tabla 14. Requisitos no funcionales	25
Tabla 15. Entradas, proceso y salidas de product_metrics.py	25
Tabla 16. Entradas, proceso y salidas de coverage_metrics.py	27
Tabla 17. Entradas, proceso y salidas de defects.py	28
Tabla 18. Entradas, proceso y salidas de density.py	29
Tabla 19. Entradas, proceso y salidas de process_metrics.py y project_metrics.py	30
Tabla 20. Entradas, proceso y salidas de estimation.py	30
Tabla 21. Entradas, proceso y salidas de cli.py	31
Tabla 22. Entradas, proceso y salidas de quality_gate.py	32
Tabla 23. Entradas, proceso y salidas de ci.yml	33
Tabla 24. Entradas, proceso y salidas de report.py	34
Tabla 25. Entradas, proceso y salidas de ai_advisor.py	35
Tabla 26. Entradas, proceso y salidas de app.py y dashboard.py	36
Tabla 27. Requisitos funcionales y pruebas automatizadas que los verifican	38
Tabla 28. Usuarios de la herramienta	39
Tabla 29. Expectativas del negocio, indicador, meta y valor real	40
Tabla 30. Actividades del plan SQA	41
Tabla 31. Pull Requests integrados a main (git log --merges --first-parent)	41
Tabla 32. Pruebas automatizadas por archivo	42
Tabla 33. Defectos de producto minados del historial	43
Tabla 34. Medidas de idoneidad funcional	45
Tabla 35. Medidas de fiabilidad	46
Tabla 36. Controles preventivos y el problema que evitan	48
Tabla 37. Ejemplos reales de trazabilidad requisito → evidencia	53
Tabla 38. Normas aplicadas, cómo y dónde	53
Tabla 39. Procesos del ciclo de vida y su práctica en el proyecto	54
Tabla 40. Decisiones del proyecto (docs/DECISIONES.md, resumidas)	54
Tabla 41. Reparto de responsabilidades entre el código y la IA	56
Tabla 42. Verificaciones de coherencia de la ejecución analizada	56
Tabla 43. Funciones más complejas	58
Tabla 44. Métricas de proceso	60
Tabla 45. Valoración por reglas de cada métrica (ai_analysis.json)	62
Tabla 46. Línea base de estimación por tarea (horas)	63
Tabla 47. Conteo de puntos de función de QualityOps AI (data/estimacion.json)	64
Tabla 48. Comparación de las técnicas	65
Tabla 49. Escenarios de prueba	66
Tabla 50. Resumen de todas las métricas (commit 9d93529)	68

1. DESCRIPCIÓN DEL PROYECTO Y JUSTIFICACIÓN

DEVOPS

En resumen: QualityOps AI es un *quality gate* inteligente para repositorios Python: mide, valida y reporta la calidad de cada cambio antes de que se integre a la rama principal. Es un proyecto DevOps porque su producto es una etapa del pipeline de integración continua (no una aplicación de negocio), y porque se desarrolló con las mismas prácticas que automatiza: ramas cortas, Pull Requests, CI obligatoria, métricas DORA y retroalimentación continua.

1.1 Qué es QualityOps AI

QualityOps AI es una herramienta de línea de comandos escrita en Python que recibe la ruta de un repositorio (`--repo`) y produce un archivo `reports/metrics.json` con todas las métricas. Sobre ese archivo trabajan tres componentes: el *quality gate*, que decide si el cambio puede integrarse; el generador del informe de calidad, que redacta un informe con formato fijo; y un *dashboard* que muestra los mismos datos de forma visual. Un componente de IA (AI Advisor) verifica la coherencia de los números y los interpreta.

Tabla 1. Ficha técnica del proyecto

Campo	Valor
Repositorio	github.com/TaikoVC/qualityops-ai (público, un solo autor)
Lenguaje y versión	Python 3.14 (probado localmente con 3.14.3 en Windows; CI en ubuntu-latest)
Tamaño del producto	1.066 KLOC: 1066 líneas de código fuente (SLOC) en 15 archivos
Funciones analizadas	87 (complejidad promedio 3.72, máxima 9)
Pruebas automatizadas	56 (pytest), 56 aprobadas
Historial	32 commits sin merge y 16 Pull Requests integrados con merge commit
Pipeline	GitHub Actions, job «calidad», en cada push y Pull Request a main
IA	Gemini (Google AI Studio, modelo gemini-3.5-flash-lite) con respaldo por reglas
Periodo de desarrollo	4 al 8 de octubre de 2026, en sprints cortos con tablero Kanban
Datos analizados	commit 9d93529, ejecución del 2026-10-08 08:41

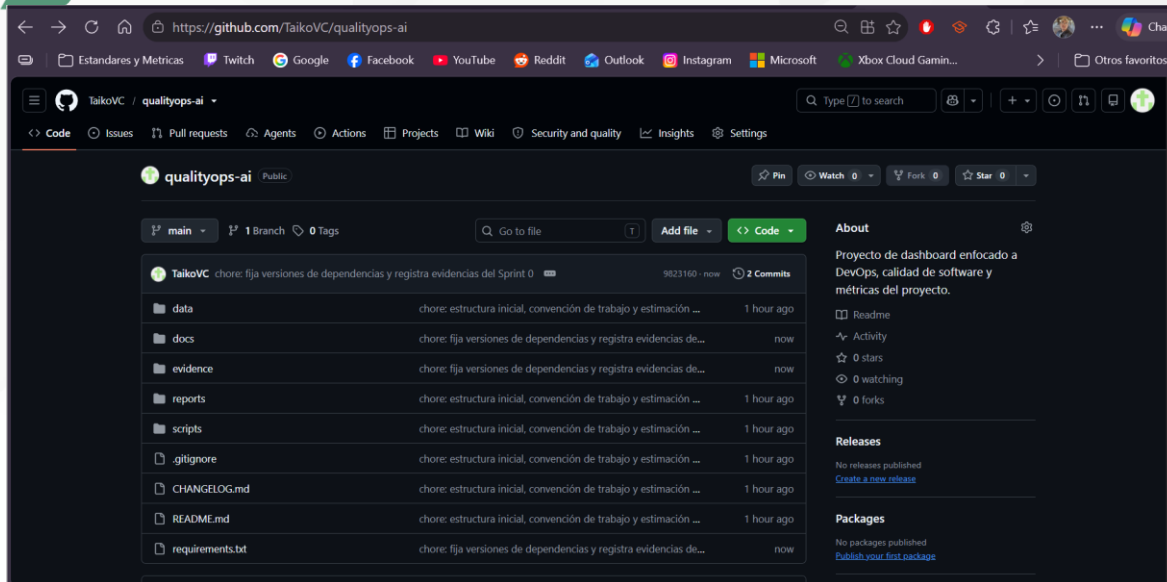


Figura 1. Repositorio público en GitHub al cierre del Sprint 0 (evidencia E01_repo_github_1)

Qué muestra. La página principal del repositorio [TaikoVC/qualityops-ai](https://github.com/TaikoVC/qualityops-ai) en GitHub durante el Sprint 0: la estructura de carpetas (`data`, `docs`, `evidence`, `reports`, `scripts`), el README y el último commit (9823160).

Qué demuestra. El proyecto vive desde el primer día en un repositorio público y versionado, con un único contribuidor (trabajo individual, DOC-08) y una estructura pensada para separar código, datos, documentación y evidencias.

1.2 Problema que resuelve y valor agregado

En un equipo de desarrollo, la calidad suele revisarse tarde: alguien corre las pruebas a mano antes de entregar, nadie mide la complejidad y los defectos se registran (si se registran) en una hoja de cálculo que nadie actualiza. El resultado es que los problemas llegan a la rama principal y se descubren cuando ya cuestan más.

QualityOps AI ataca ese problema con **validación preventiva automática**:

- **Mide en cada cambio**, no al final: el pipeline corre en cada Pull Request.
- **Bloquea antes de integrar**: si la cobertura baja del 75 %, alguna función supera complejidad 10 o falla una prueba, GitHub no permite el merge.
- **Obtiene los defectos del historial de Git**, no de un registro manual: cualquiera puede verificar los datos con `git log`.

- **Explica los resultados en lenguaje natural** con un modelo de lenguaje, pero sin dejar que la IA calcule las cifras.
- **Es integrable a cualquier repositorio Python con Git** (opción `--repo`), sin modificar el proyecto analizado.

1.3 Justificación como proyecto DevOps

La actividad pide un programa «enfocado a las tecnologías en DevOps» y que el proyecto se justifique. DevOps es la unión de desarrollo y operación mediante automatización, integración continua, medición y retroalimentación rápida (Forsgren et al., 2018). Un sistema de restaurante o una calculadora usarían DevOps solo como medio; en QualityOps AI **la práctica DevOps es el producto**. La siguiente tabla relaciona cada práctica con su implementación y su evidencia.

Tabla 2. Prácticas DevOps y cómo se implementan en QualityOps AI

Práctica DevOps	Implementación en el proyecto	Evidencia
Integración continua (CI)	Workflow <code>.github/workflows/ci.yml</code> en cada push y Pull Request a main	E08, E08d, E10k
Pipeline como código	El pipeline completo está versionado en el repositorio y se revisa por PR	ci.yml, diagrama del pipeline (cap. 3)
Quality gates / shift-left	<code>quality_gate.py</code> termina con código 1 si no se cumplen los umbrales	E08a, E17
Protección de la rama principal	Ruleset <code>proteger-main</code> : PR obligatorio, check «calidad» obligatorio, sin force push	E08b
Trunk-based con ramas cortas	Una rama por tarea; 16 PRs integrados con merge commit	Tabla de PRs (cap. 8)
Métricas DORA	MTTR (tiempo de restauración) y frecuencia de integración a partir del historial	Capítulo 12
Monitoreo y retroalimentación	Resumen en Actions, artefacto <code>reportes-calidad</code> , dashboard	E08c, E09
Infraestructura reproducible	Dependencias con versión fija y Python fijado en CI	requirements.txt, E00b
Gestión de secretos	Llave de Gemini como secreto del repositorio, nunca en el código	ci.yml, D19

1.4 Tecnologías y frameworks

Elegí herramientas estándar, gratuitas y explicables. Cada una cumple una función que se puede señalar en el código o en el pipeline.

Tabla 3. Tecnologías utilizadas

Tecnología	Versión	Para qué se usa
Python	3.14	Lenguaje del programa y de las pruebas
pytest / pytest-cov	9.1.1 / 7.1.0	Ejecución de pruebas y medición de cobertura de líneas
radon	6.0.1	Complejidad ciclomática por función y conteo de líneas (LOC, SLOC)
ruff	0.16.10	Revisión automática de estilo y errores (linter) en local y en CI
Streamlit	1.65.0	Framework del dashboard web (6 pestañas)

Tecnología	Versión	Para qué se usa
Git y GitHub	2.53	Control de versiones, Pull Requests, ruleset, issues y Projects (Kanban)
GitHub Actions	checkout@v5, setup-python@v6	Pipeline de integración continua
Gemini API (Google AI Studio)	gemini-3.5-flash-lite	Interpretación de métricas en lenguaje natural (AI Advisor)
Graphviz / matplotlib	—	Diagramas y gráficas de este reporte (no son dependencias del producto)

2. METODOLOGÍA DE TRABAJO

En resumen: Trabajé con Kanban en GitHub Projects, en sprints de un día, con una rama y un Pull Request por tarea. Antes de escribir código registré la estimación de cada tarea; al cerrarla registré su tiempo real. Los defectos se registran en el propio mensaje del commit que los corrige (trailers), de modo que el historial de Git es la fuente de datos de las métricas. La IA (Claude) me asistió para generar código, y yo ejecuté y verifiqué cada salida.

2.1 Entorno de trabajo

Desarrollé en Windows con PowerShell, Python 3.14.3 y Git 2.53, dentro de un entorno virtual (`.venv`). Las dependencias tienen versión fija en `requirements.txt` para que la ejecución en GitHub Actions use exactamente las mismas versiones que mi equipo.

```
PS C:\WINDOWS\system32> python --version
Python 3.14.3
PS C:\WINDOWS\system32> git --version
git version 2.53.0.windows.1
PS C:\WINDOWS\system32> git config --global user.name
Ivan Valle
PS C:\WINDOWS\system32> git config --global user.email
ivan_vallec_13@hotmail.com
PS C:\WINDOWS\system32> mkdir qualityops-ai; cd qualityops-ai

Directorio: C:\WINDOWS\system32

Mode                LastWriteTime         Length Name
----                -
d-----          04/10/2026   09:39 p. m.         qualityops-ai

PS C:\WINDOWS\system32\qualityops-ai> git init -b main
Initialized empty Git repository in C:/Windows/System32/qualityops-ai/.git/
PS C:\WINDOWS\system32\qualityops-ai> python -m venv .venv
PS C:\WINDOWS\system32\qualityops-ai> .\.venv\Scripts\Activate.ps1
(.venv) PS C:\WINDOWS\system32\qualityops-ai> python -m pip install --upgrade pip
Requirement already satisfied: pip in c:\windows\system32\qualityops-ai\.venv\lib\site-packages (25.3)
Collecting pip
  Downloading pip-26.2.1-py3-none-any.whl.metadata (4.6 kB)
  Downloading pip-26.2.1-py3-none-any.whl (1.8 MB)
----- 1.8/1.8 MB 7.2 MB/s 0:00:00
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 25.3
    Uninstalling pip-25.3:
      Successfully uninstalled pip-25.3
  Successfully installed pip-26.2.1
(.venv) PS C:\WINDOWS\system32\qualityops-ai> _
```

Figura 2. Creación del entorno: versiones de Python y Git, git init y entorno virtual (evidencia E00_entorno)

Qué muestra. Las versiones instaladas (Python 3.14.3, Git 2.53), la identidad configurada en Git, la creación del repositorio con `git init -b main` y del entorno virtual `.venv`.

Qué demuestra. El punto de partida del proyecto (4 de octubre, 21:39). La carpeta inicial quedó en `C:\WINDOWS\system32` porque abrí PowerShell como administrador; lo corregí moviendo el repositorio a `C:\Users\LPC\proyectos` (decisión D10).

```

Downloading pygments-2.21.0-py3-none-any.whl (1.3 MB)
----- 1.3/1.3 MB 14.9 MB/s 0:00:00
Downloading python-dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)
Downloading referencing-0.37.0-py3-none-any.whl (26 kB)
Downloading rpds_py-2026.9.1-cp314-cp314-win_amd64.whl (222 kB)
Downloading six-1.17.0-py2.py3-none-any.whl (11 kB)
Downloading tzdata-2026.5-py2.py3-none-any.whl (347 kB)
Installing collected packages: websockets, watchdog, urllib3, tzdata, typing-extensions, toml, six, ruff, rpds-py, python-multipart, pygments, pyarrow, protobuf, pluggy, pillow, packaging, numpy, narwhals, MarkupSafe, itsdangerous, iniconfig, idna, httptools, h11, coverage, colorama, click, charset-normalizer, certifi, attrs, uvicorn, requests, referencing, python-dateutil, pytest, mando, Jinja2, anyio, starlette, radon, pytest-cov, pydeck, pandas, jsonschema-specifications, jsonschema, altair, streamlit
Successfully installed MarkupSafe-3.0.4 altair-6.3.0 anyio-4.15.1 attrs-26.1.0 certifi-2026.7.22 charset-normalizer-3.5.2 click-8.5.0 colorama-0.4.6 coverage-7.16.2 h11-0.16.0 httptools-0.8.0 idna-3.20 iniconfig-2.3.0 itsdangerous-2.2.0 Jinja2-3.1.6 jsonschema-4.26.0 jsonschema-specifications-2025.9.1 mando-0.7.1 narwhals-2.26.0 numpy-2.5.3 packaging-26.3 pandas-3.0.6 pillow-12.3.0 pluggy-1.6.0 protobuf-7.36.2 pyarrow-25.0.1 pydeck-0.9.3 pygments-2.21.0 pytest-9.1.1 pytest-cov-7.1.0 python-dateutil-2.9.0.post0 python-multipart-0.0.32 radon-6.0.1 referencing-0.37.0 requests-2.34.2 rpds-py-2026.9.1 ruff-0.16.10 six-1.17.0 starlette-1.7.0 streamlit-1.65.0 toml-0.10.2 typing-extensions-4.16.0 tzdata-2026.5 urllib3-2.8.0 uvicorn-0.54.0 watchdog-6.0.0 websockets-17.2

[notice] A new release of pip is available: 25.3 -> 26.2.1
[notice] To update, run: python.exe -m pip inst
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> pip freeze | Select-String "^(pytest|pytest-cov|radon|ruff|streamlit)=="

pytest==9.1.1
pytest-cov==7.1.0
radon==6.0.1
ruff==0.16.10
streamlit==1.65.0

(.venv) PS C:\Users\LPC\proyectos\qualityops-ai>

```

Figura 3. Dependencias instaladas y versiones resultantes (evidencia E00b_pip_install_2)

Qué muestra. El final de `pip install -r requirements.txt` y el filtro de `pip freeze` con las cinco dependencias directas: `pytest 9.1.1`, `pytest-cov 7.1.0`, `radon 6.0.1`, `ruff 0.16.10` y `streamlit 1.65.0`.

Qué demuestra. Las versiones son las mismas que fija `requirements.txt`; por eso los resultados locales y los del CI son comparables (requisito de reproducibilidad RNF-01).

2.2 Kanban y sprints

El tablero «QualityOps AI - Kanban» de GitHub Projects tiene las columnas Backlog, Ready, In Progress (límite de trabajo en curso = 2), Review/Validate y Done. Creé una *issue* por tarea (T01 a T25) con el script `scripts/crear_issues.ps1`, que lee `data/time_log.csv`; así el tablero y la estimación tienen la misma fuente.

Tabla 4. Sprints planeados (pyproject.toml) y tareas asignadas (data/time_log.csv)

Sprint	Fecha planeada	Tareas
S0	2026-10-04	T01, T02, T03
S1A	2026-10-05	T04, T05
S1B	2026-10-05	T06, T07, T08, T09
S1C	2026-10-05	T10, T11
S1D	2026-10-05	T12, T20
S2A	2026-10-06	T13, T14
S2B	2026-10-06	T15, T16
S2C	2026-10-06	T17
S2D	2026-10-06	T18, T19
S3A	2026-10-07	T22, T26
S3B	2026-10-07	T21

Sprint	Fecha planeada	Tareas
S3C	2026-10-07	T23, T24, T25

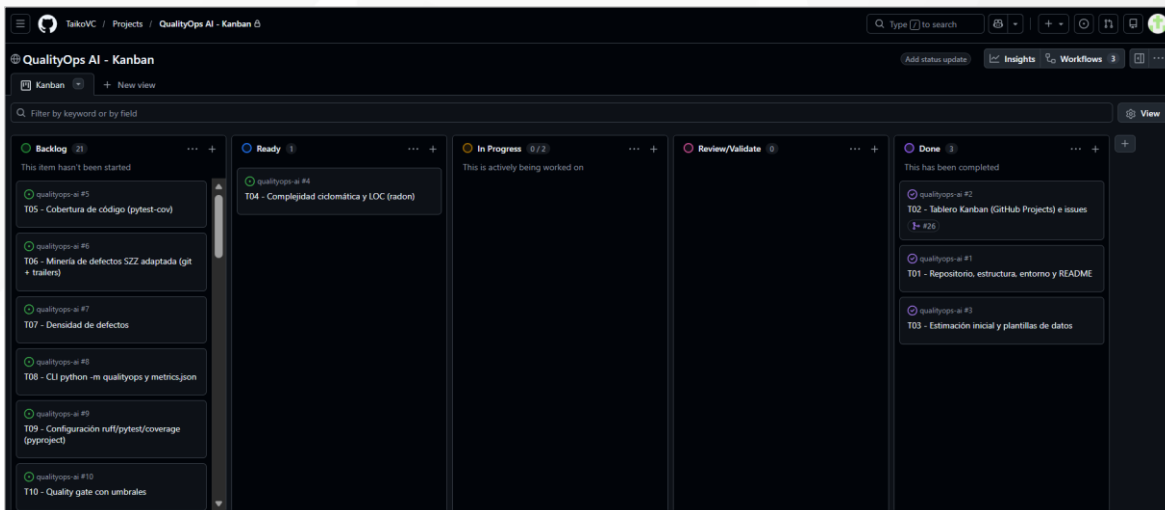


Figura 4. Tablero Kanban al cierre del Sprint 0 (evidencia E02c_kanban)

Qué muestra. El tablero con T01, T02 y T03 en *Done* (T02 enlazada al PR #26), T04 en *Ready* y el resto en *Backlog*; la columna *In Progress* muestra el límite 0/2.

Qué demuestra. El flujo Kanban con límite de trabajo en curso y la trazabilidad tarea → issue → Pull Request (requisitos PRO-01 y CAL-08).

2.3 Convenciones de trabajo

Para que el historial de Git sea medible, todos los commits siguen **Conventional Commits 1.0.0**: el tipo (`feat`, `fix`, `docs`, `chore`, `ci`, `test`) va al inicio del mensaje. Cada tarea vive en su propia rama y se integra con un Pull Request y un *merge commit* (nunca se reescribe la historia), y una etiqueta `vx.y.z` marca una versión en producción.

Tabla 5. Commits del proyecto por tipo (git log, sin merges)

Tipo	Significado	Commits
feat	Funcionalidad nueva	12
docs	Documentación y registros	12
fix	Corrección de un defecto	4
chore	Mantenimiento (estructura, dependencias)	3
ci	Pipeline	1
Total		32

Cada corrección (`fix:`) lleva además **trailers**, líneas al final del mensaje que la herramienta lee automáticamente: `Severidad` (critica, mayor o menor), `Detectado-en` (revision, pruebas o produccion), `Evidencia` (archivo de la captura) y `Refs` (issue).

Ejemplo real: commit a6139f9 (defecto 1)

fix(ai_advisor): nombres de archivo con guion bajo se mostraban en negritas en el informe

Severidad: menor

Detectado-en: revision

Evidencia: evidence/E15_defecto1_markdown_main_py_PR38_2026-10-08_0120.png

Refs: #17

```
no checks reported on the T12-T17-informe-ia branch
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> git log -1
commit a6139f9c1bdc4299da0e29b7b56acc6c2d6a73ef (HEAD -> T12-T17-informe-ia, origin/T12-T17-informe-ia)
Author: Ivan Valle <ivan_vallec_13@hotmail.com>
Date: Thu Oct 8 01:31:31 2026 -0600

    fix(ai_advisor): nombres de archivo con guion bajo se mostraban en negritas en el informe

    Severidad: menor
    Detectado-en: revision
    Evidencia: evidence/E15_defecto1_markdown_main_py_PR38_2026-10-08_0120.png (resumen de Actions del PR #38)
    Refs: #17
```

Figura 5. Commit de corrección con sus trailers en git log (evidencia E15c)

Qué muestra. La salida de `git log -1` del commit a6139f9 con autor, fecha (8 de octubre, 01:31) y los cuatro trailers.

Qué demuestra. El registro del defecto no está en una hoja aparte: está en el historial, con fecha exacta. De ahí salen la severidad, la fase de detección, el MTDD y el MTTR del capítulo 12.

2.4 Fuentes de datos y regla de evidencia

Ninguna métrica se captura a mano. La siguiente tabla muestra de dónde sale cada dato y quién lo produce.

Tabla 6. Fuentes de datos del proyecto

Dato	Fuente	Lo produce
Complejidad, LOC, SLOC	Código fuente *.py	radon, dentro de product_metrics.py
Pruebas y cobertura	Ejecución de pytest	pytest-cov, dentro de coverage_metrics.py
Defectos, severidad, fase	Historial de Git (commits fix: y trailers)	defects.py (algoritmo SZZ)
MTDD, MTTR, eficacias	Fechas de commits inductor, fix y merge	process_metrics.py, project_metrics.py
Tiempos estimados y reales	data/time_log.csv (estimación en el commit 76f14fb, antes del código)	El autor, por tarea
Datos de estimación	data/estimacion.json (conteo de PF y proyecto de referencia)	El autor
Umbral y calendario	pyproject.toml	El autor (configuración)
Capturas	evidence/E##_descripcion_fecha.png + docs/EVIDENCIAS.md	El autor, en cada tarea

2.5 Uso de IA durante el desarrollo

Usé Claude (Anthropic) como copiloto técnico: analizó los requisitos, propuso la arquitectura y generó código y pruebas. La regla fue «la IA propone; el autor

ejecuta, verifica la salida real y decide». Cada uso quedó registrado en `docs/AI_LOG.md`, incluidos los errores de la propia IA.

Tabla 7. Bitácora de uso de IA (`docs/AI_LOG.md`, resumida)

Fecha	Sprint	Para qué se usó	Qué se rechazó o corrigió
2026-10-04	S0	Análisis del plan maestro: requisitos, matriz, riesgos, ambigüedades, backlog, arquitectura	—
2026-10-04	S0	Validación de la herramienta <code>metricas</code> y de dos reportes anteriores antes de reutilizarlos	Textos de interpretación fijos de <code>secrets-engine</code> ; versiones de normas desactualizadas (25010:2011, 90003:2004, IEEE 829-2008)
2026-10-05	S1A	T04: generar <code>product_metrics.py</code> y sus pruebas	Primera versión tenía <code>analizar_complejidad</code> con CC 11 (> 10); se dividió antes de entregarla
2026-10-05	S1A	T05: generar <code>coverage_metrics.py</code> y sus pruebas	ruff (PLW1510, RUF010) marcó dos detalles; se corrigieron antes de entregar
2026-10-05	S1B	T06: adaptar la minería SZZ de la herramienta <code>metricas</code> a Python + trailers	En el entorno de la IA falló la prueba del commit raíz (<code>blame</code> marca el límite con ^); se cambió a <code>--porcelain.minar_defectos</code> tenía CC 17; se dividió
2026-10-06	S1B–S1C	T07–T10: densidad, CLI, configuración y quality gate	ruff marcó detalles de estilo que se corrigieron antes de entregar
2026-10-08	S1C	T11: workflow de GitHub Actions y resumen en Actions	—
2026-10-08	S2A	T13/T14: métricas de proceso y proyecto	Las etiquetas de sprint escritas antes no reflejaban el retraso real; se corrigió midiendo por fecha
2026-10-08	S2B–S2C	T15/T12/T17: estimación, informe 29119-3 y AI Advisor	La IA no calcula cifras: solo interpreta y verifica coherencia; la ruta LLM queda sin probar con API real por falta de llave

El caso más importante fue un **error de la IA**: propuso usar GitHub Models como proveedor del modelo de lenguaje, basándose en una fuente desactualizada. La ejecución real en el CI devolvió una respuesta vacía; la documentación oficial confirmó que ese servicio se retiró el 30 de julio de 2026. Lo registré como defecto real (mayor, detectado en pruebas), lo corregí y cambié a Gemini (decisiones D18 y D19). Este episodio muestra por qué la regla de verificar cada salida es necesaria.

3. ARQUITECTURA, EJECUCIÓN Y REPRODUCCIÓN

En resumen: El programa tiene tres capas: un motor de métricas determinista (siete módulos que alimentan una CLI), un archivo `metrics.json` que es el único contrato entre capas, y una capa de validación y presentación (quality gate, informe, AI Advisor y dashboard). En GitHub Actions las capas se ejecutan en siete pasos. Todo se reproduce con cinco comandos.

3.1 Arquitectura de componentes

La siguiente figura muestra los componentes reales del programa y sus dependencias, tal como aparecen en los `import` del código. Las entradas (arriba) son todo lo que el programa lee del repositorio analizado; el motor (en azul) calcula las métricas sin IA; `metrics.json` (en amarillo) es el punto de unión; y la capa verde valida, interpreta y presenta.

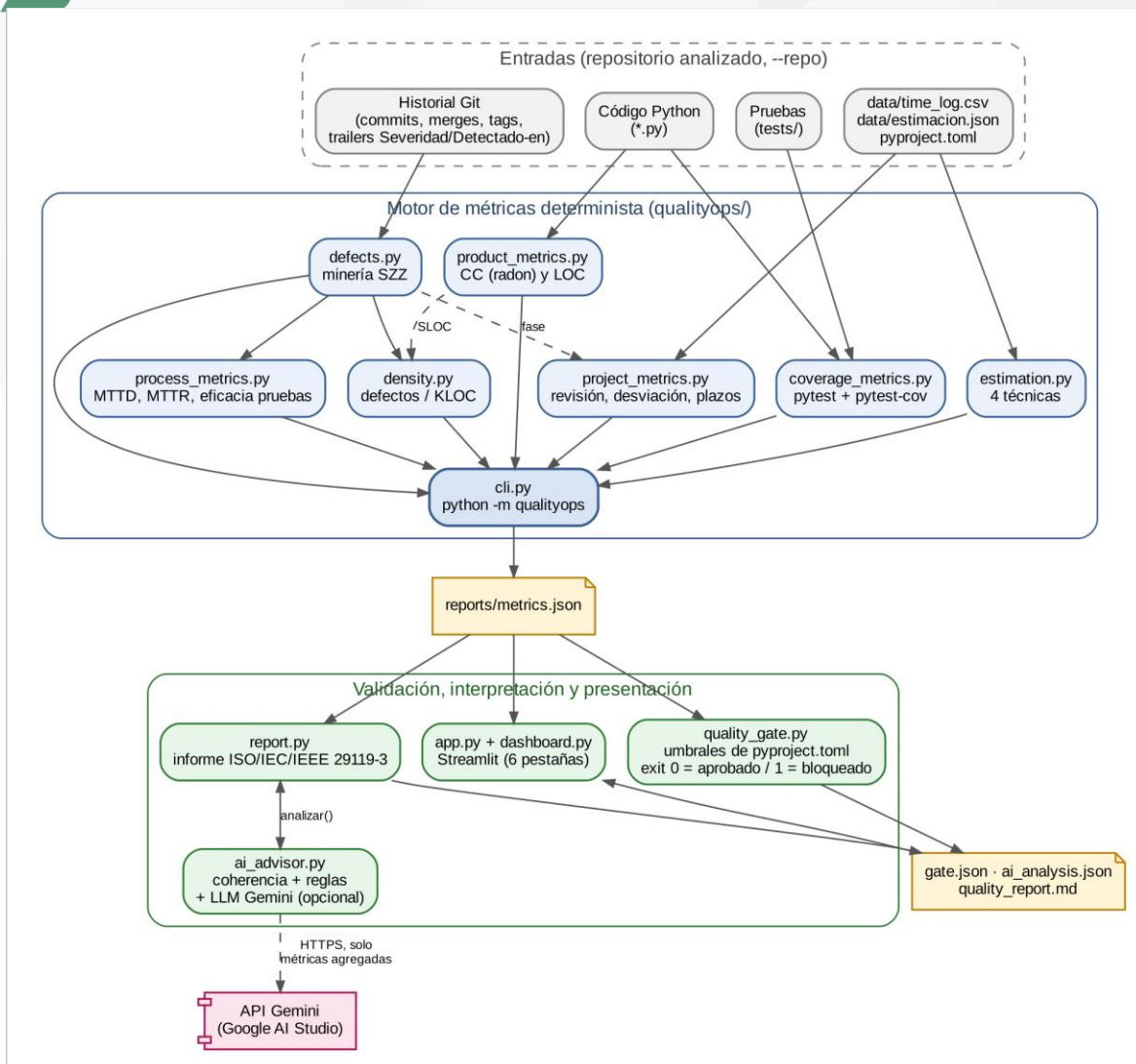


Figura 6. Arquitectura de componentes de QualityOps AI

Cómo leerla. Cada caja azul es un archivo de `qualityops/`. Las flechas indican quién usa a quién: por ejemplo, `density.py` usa a `defects.py` (cuántos defectos) y a `product_metrics.py` (cuántas líneas). `cli.py` reúne todo en `metrics.json`. La única conexión externa es la flecha punteada hacia la API de Gemini, que solo recibe métricas agregadas (nunca código ni datos personales).

Por qué está diseñada así. Separar el cálculo (determinista) de la interpretación (IA) hace que las cifras sean reproducibles: si la IA falla o no está configurada, los números no cambian y el informe se genera por reglas (decisiones D02 y D03).

Cómo reproducirla. `dot -Tpng docs/diagramas/arquitectura.dot -o arquitectura.png.`

Tabla 8. Módulos del producto con sus métricas (commit 9d93529)

Archivo	SLOC	Funciones	CC máx.	Cobertura %
app.py	76	0	—	85.29
__init__.py	1	0	—	100.0
__main__.py	3	0	—	0.0
ai_advisor.py	170	15	9	89.76
cli.py	73	5	2	100.0
coverage_metrics.py	91	6	7	80.6
dashboard.py	37	5	8	100.0
defects.py	163	15	9	83.87
density.py	37	4	4	61.54
estimation.py	51	7	9	100.0
process_metrics.py	27	3	7	100.0
product_metrics.py	95	7	6	80.33
project_metrics.py	61	8	7	92.73
quality_gate.py	69	5	6	97.83
report.py	112	7	9	97.37

3.2 Pipeline de integración continua

El archivo `.github/workflows/ci.yml` define un solo job, «calidad», que se ejecuta en cada push a `main` y en cada Pull Request. La siguiente figura muestra sus siete pasos y lo que ocurre después según el resultado del quality gate.

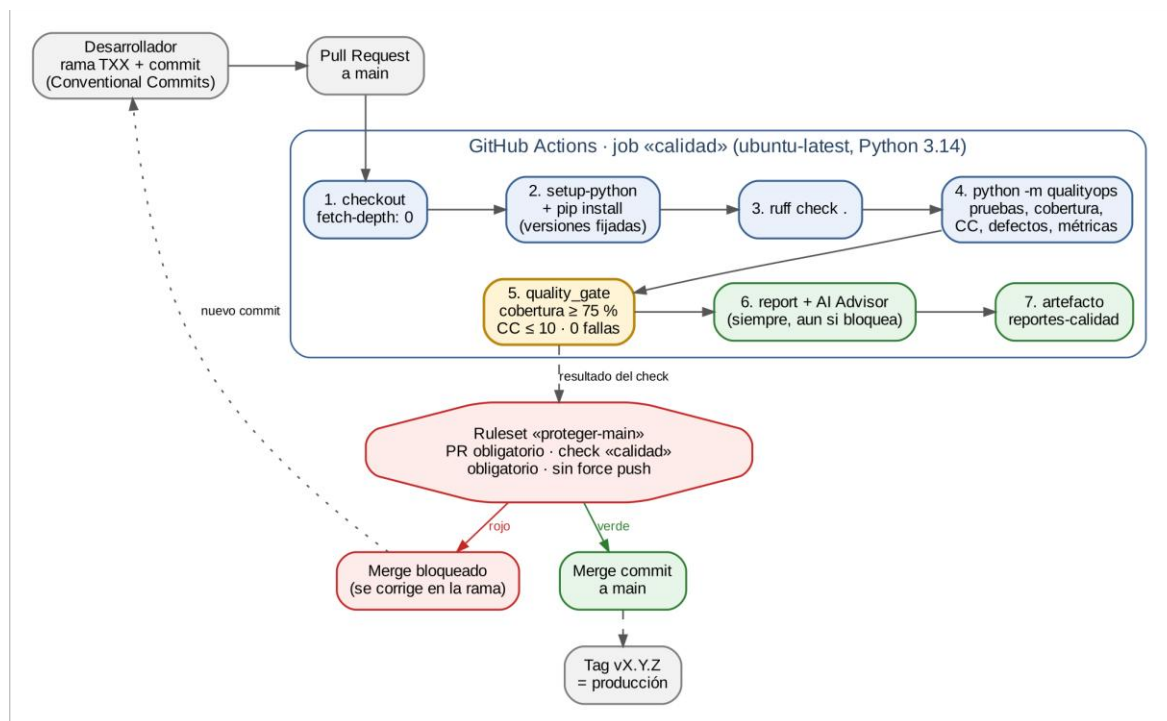


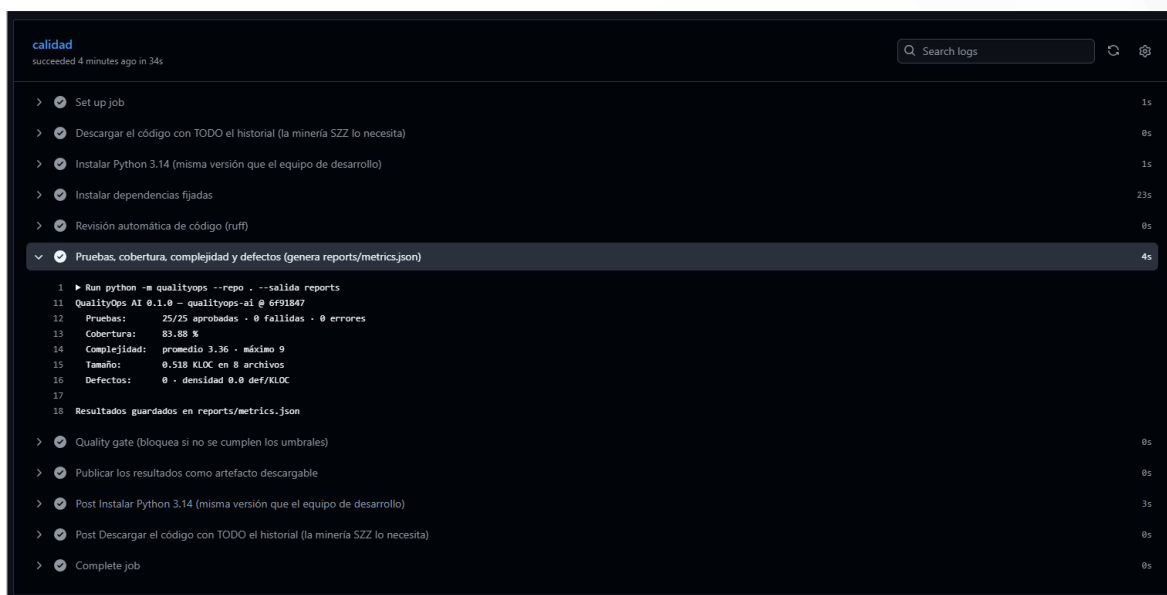
Figura 7. Pipeline CI de QualityOps AI y regla de protección de main

Cómo leerla. El desarrollador abre un Pull Request; GitHub ejecuta los pasos 1 a 7. El paso 5 (quality gate) decide el color del check. El ruleset `proteger-main` exige

que ese check esté en verde: si está en rojo, el merge queda bloqueado y la corrección se hace en la misma rama con un nuevo commit.

Detalles importantes. El paso 1 descarga todo el historial (`fetch-depth: 0`) porque la minería de defectos lo necesita. El paso 6 se ejecuta siempre (`if: always()`), aun cuando el gate bloquea, para que el informe explique por qué se bloqueó. El paso 7 publica los reportes como artefacto descargable.

Relación con ISO/IEC/IEEE 12207. Los pasos 3 a 5 corresponden a los procesos de verificación; el ruleset y los merge commits a gestión de configuración; la etiqueta de versión al proceso de liberación (ver capítulo 11).



```

calidad
succeeded 4 minutes ago in 34s

> Set up job 1s
> Descargar el código con TODO el historial (la minería SZZ lo necesita) 0s
> Instalar Python 3.14 (misma versión que el equipo de desarrollo) 1s
> Instalar dependencias fijadas 23s
> Revisión automática de código (ruff) 0s
> Pruebas, cobertura, complejidad y defectos (genera reports/metrics.json) 4s
  1 ▶ Run python -m qualityops --repo . --salida reports
 11 QualityOps AI 0.1.0 - qualityops-ai @ 6f91847
 12 Pruebas: 25/25 aprobadas - 0 fallidas - 0 errores
 13 Cobertura: 83.88 %
 14 Complejidad: promedio 3.36 - máximo 9
 15 Tamaño: 0.518 KLOC en 8 archivos
 16 Defectos: 0 - densidad 0.0 def/KLOC
 17
 18 Resultados guardados en reports/metrics.json
> Quality gate (bloquea si no se cumplen los umbrales) 0s
> Publicar los resultados como artefacto descargable 0s
> Post Instalar Python 3.14 (misma versión que el equipo de desarrollo) 3s
> Post Descargar el código con TODO el historial (la minería SZZ lo necesita) 0s
> Complete job 0s
  
```

Figura 8. Log del job «calidad» en GitHub Actions, paso a paso (evidencia E08d)

Qué muestra. La ejecución del PR #35 (T11): cada paso con su duración y, expandido, el paso de métricas con la salida de la CLI (25/25 pruebas, cobertura 83.88 %, complejidad máxima 9, 0.518 KLOC).

Qué demuestra. La automatización es real y visible: el mismo comando que corro en mi equipo se ejecuta en un servidor limpio de GitHub en 34 segundos (PRG-03).

3.3 Cómo reproducir los resultados

Cualquier persona con Python 3.14 y Git puede repetir todos los resultados de este reporte en Windows (PowerShell). En Linux o macOS solo cambia la activación del entorno (`source .venv/bin/activate`).

Reproducción completa (PowerShell)

```
# 1. Obtener el código con todo su historial
git clone https://github.com/TaikoVC/qualityops-ai.git
cd qualityops-ai
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt

# 2. Calcular métricas, aplicar el gate y generar el informe
python -m qualityops --repo . --salida reports
python -m qualityops.quality_gate --metricas reports/metrics.json
python -m qualityops.report --metricas reports/metrics.json

# 3. (Opcional) interpretación con LLM: la llave no se guarda en ningún archivo
$env:GEMINI_API_KEY = Read-Host "Pega tu llave (no se guarda)"
python -m qualityops.report

# 4. Dashboard
streamlit run app.py
```

3.4 Guía de comandos

Tabla 9. Comandos del programa

Comando	Qué hace	Salida
<code>python -m qualityops --repo R --salida S</code>	Ejecuta todo el motor sobre el repositorio R	S/metrics.json y resumen en terminal
<code>python -m qualityops.quality_gate</code>	Compara metrics.json con los umbrales de pyproject.toml	gate.json; código 0 = aprobado, 1 = bloqueado
<code>python -m qualityops.report</code>	Informe ISO/IEC/IEEE 29119-3 + AI Advisor	quality_report.md, ai_analysis.json
<code>python -m qualityops.product_metrics --repo R</code>	Solo complejidad y líneas	Tabla por función
<code>python -m qualityops.coverage_metrics --repo R</code>	Solo pruebas y cobertura	Tabla por archivo
<code>python -m qualityops.defects --repo R</code>	Solo minería de defectos	Lista de defectos con MTMD/MTTR
<code>python -m qualityops.density --repo R</code>	Solo densidad de defectos	Tabla por archivo y global
<code>streamlit run app.py</code>	Dashboard web en localhost:8501	6 pestañas
<code>python -m pytest -q / ruff check .</code>	Pruebas del propio programa / linter	Resultado de pytest / ruff

3.5 Ejecución y salida real

La siguiente figura es la salida de la ejecución completa en mi equipo: la CLI imprime el resumen de todas las métricas, el quality gate evalúa sus tres criterios y el informe se genera con la interpretación del modelo de lenguaje.

```
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> python -m qualityops --repo . --salida reports
QualityOps AI 0.1.0 - qualityops-ai @ 9d93529
Pruebas: 56/56 aprobadas · 0 fallidas · 0 errores
Cobertura: 88.73 %
Complejidad: promedio 3.72 · máximo 9
Tamaño: 1.066 KLOC en 15 archivos
Defectos: 3 · densidad 2.81 def/KLOC
MTTD / MTTR: 0.23 h / 0.07 h (n = 3 / 3)
Desviación: -61.97 % (18 tareas terminadas)
Plazos: 33.33 % a tiempo
Estimación: juicio 22.5 h · PERT 24.79 h · PF 4.24 h · análoga 4.79 h

Resultados guardados en reports\metrics.json
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> python -m qualityops.quality_gate --metricas reports/metrics.json

Quality gate - QualityOps AI
[OK ] Pruebas automatizadas sin fallas: 56/56 aprobadas (umbral total > 0 y 0 fallidas/errores)
[OK ] Cobertura de código: 88.73 (umbral >= 75.0 %)
[OK ] Complejidad ciclomática máxima: 9 (umbral <= 10)

Dictamen: APROBADO

(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> python -m qualityops.report
Informe generado: reports\quality_report.md · dictamen: LIBERAR CON CONDICIONES · interpretación: llm
```

Figura 9. Ejecución final: CLI, quality gate e informe con LLM (evidencia E16a)

Qué muestra. Los tres comandos del paso 2 de la sección 3.3 sobre la versión final (commit 9d93529): 56/56 pruebas, cobertura 88.73 %, complejidad promedio 3.72 (máx. 9), 1.066 KLOC en 15 archivos, 3 defectos (2.81 def/KLOC), MTTD/MTTR 0.23/0.07 h, desviación -61.97 % (18 tareas), plazos 33.33 % y las cuatro estimaciones. El gate aprueba sus tres criterios y el informe se genera con `interpretación: llm`.

Qué demuestra. El programa funciona de extremo a extremo y produce salida visible (PRG-06).

4. CALIDAD DE SOFTWARE Y GRADO DE CUMPLIMIENTO

En resumen: Defino la calidad como el grado en que el programa cumple requisitos explícitos y verificables. Los mido con una matriz viva de 44 requisitos derivados de la actividad: un requisito solo pasa a «Cumple» cuando tiene evidencia real. Con este reporte, **39 de 44 (88.6 %) cumplen** y 5 están en progreso (PRG-09, CAL-05, MET-R3, PRO-01, PRO-03); ninguno está en «No cumple».

4.1 Definición de calidad de software

La norma ISO/IEC 25010:2023 define la calidad de un producto de software como el grado en que satisface las necesidades declaradas e implícitas de sus partes interesadas, y la organiza en características medibles (idoneidad funcional, fiabilidad, mantenibilidad, entre otras). IEEE 730-2014 agrega la visión de proceso: la calidad se **asegura** con actividades planificadas que dan confianza de que el producto cumple sus requisitos.

De ambas definiciones tomo dos ideas para el proyecto: (1) la calidad se expresa contra requisitos concretos, no como opinión; y (2) cada afirmación de calidad necesita evidencia que un tercero pueda revisar. Por eso el grado de cumplimiento se calcula, no se declara.

4.2 Cómo se mide el grado de cumplimiento

[docs/MATRIZ_CUMPLIMIENTO.md](#) lista cada requisito de la actividad con un ID, su implementación, su evidencia, su criterio de aceptación y un estado. Los estados son: ☒ Cumple, ☐ En progreso, ☐ Parcial, ☐ Pendiente y ☒ No cumple. El grado de cumplimiento es:

$$\text{Grado de cumplimiento (\%)} = \frac{\text{requisitos en estado Cumple}}{\text{total de requisitos}} \times 100$$

Este cálculo no lo hago a mano: la función `resumen_matriz()` de [qualityops/dashboard.py](#) lee el archivo y cuenta los estados, y el dashboard lo muestra. Así, actualizar la matriz actualiza el indicador.

4.3 Resultado del grado de cumplimiento

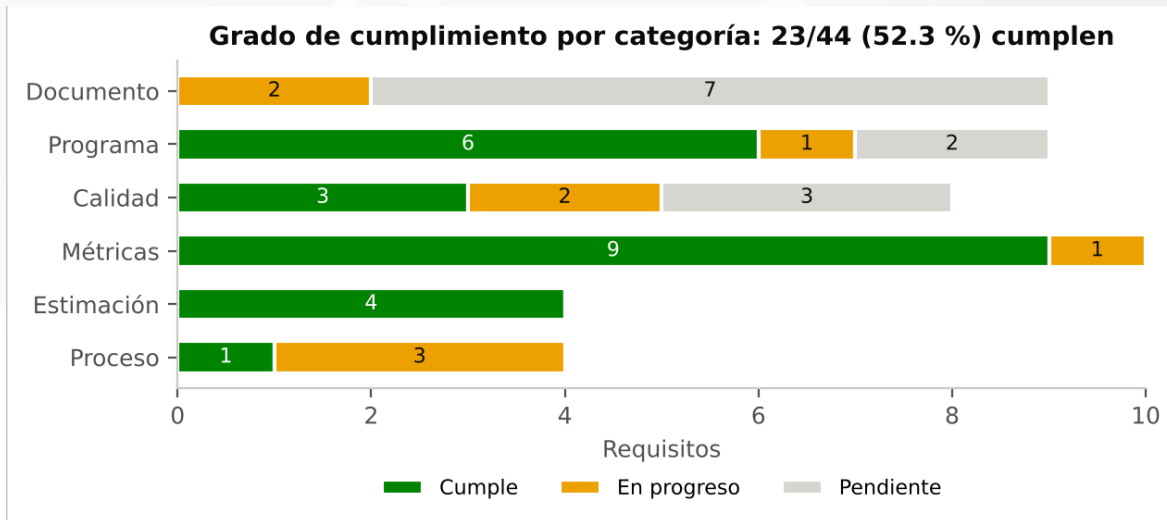


Figura 10. Estado de los requisitos de la matriz viva por categoría

Qué muestra. Los 44 requisitos agrupados por categoría (Documento, Programa, Calidad, Métricas, Estimación y Proceso) y coloreados por estado. Verde = Cumple, amarillo = En progreso, gris = Pendiente.

Cómo leerla. Las barras verdes son requisitos con evidencia. Los de **Documento** y varios de **Calidad** se cumplen con este mismo reporte (portada, índice, requisitos, ISO 25010, carácter sistemático), por eso su evidencia es el reporte (E13). Lo que queda en amarillo se explica en la sección 4.4.

Cómo se generó. `python docs/reporte/graficas.py` lee `docs/MATRIZ_CUMPLIMIENTO.md` y cuenta los estados.



Figura 11. Grado de cumplimiento calculado por el dashboard (evidencia E09b)

Qué muestra. La pestaña «Resumen y cumplimiento» del dashboard con el indicador «Requisitos que cumplen» y la gráfica por estado, calculados en el momento de la captura (commit 1675934, 8 de octubre a las 02:15).

Qué demuestra. El porcentaje lo calcula el programa a partir de la matriz (CAL-01). La captura es anterior al registro de T18/T19: en ese momento eran 15 de 44; la matriz actual da el valor del recuadro inicial.

Tabla 10. Matriz de cumplimiento completa (docs/MATRIZ_CUMPLIMIENTO.md)

ID	Requisito	Evidencia	Estado
DOC-01	Portada	Reporte final (E13): portada	Cumple
DOC-02	Introducción	Reporte final (E13): I. Introducción	Cumple
DOC-03	Índice tabulado	Reporte final (E13): II. Índice	Cumple
DOC-04	Imágenes explicadas	Reporte final (E13), E03	Cumple
DOC-05	Conclusión	Reporte final (E13): III. Conclusión	Cumple
DOC-06	≥ 10 hojas con tablas e imágenes	Reporte final (E13): cap. 6	Cumple
DOC-07	Plantilla UTCJ + referencias	Reporte final (E13): IV. Referencias (APA)	Cumple
DOC-08	Trabajo individual	E01; git shortlog -sne: Ivan Valle (commits) y TaikoVC (su cuenta de GitHub, merges)	Cumple
DOC-09	Normas aplicadas, claras y puntuales	Reporte final (E13): cap. 11.3	Cumple
PRG-01	Tecnologías DevOps	E08, E08b, E08e, PR #35	Cumple
PRG-02	IA en frameworks	E10h, E10i, E10k, PR #40, D19	Cumple
PRG-03	Automatización	E08, E08c, E08d	Cumple
PRG-04	Validación de software	E05, E05b, E10h, E10k	Cumple
PRG-05	GitHub + repositorio	E00, E00c, E01, README	Cumple
PRG-06	Ejecución y salida	E04b, E09a–h, E10h	Cumple
PRG-07	Métricas	E16a (todas las métricas en la ejecución final, commit 9d93529)	Cumple
PRG-08	Proyecto DevOps justificado	Reporte final (E13): cap. 1.3	Cumple
PRG-09	Revisión en clase	Reporte final (E13): cap. 14	En progreso (guion listo; ensayo pendiente)
CAL-01	Calidad y grado de cumplimiento	E09a, E09b	Cumple
CAL-02	Informe de calidad en formato	E10, E15b	Cumple
CAL-03	Requisitos explícitos	Reporte final (E13): cap. 6	Cumple
CAL-04	Requisitos de usuario y negocio	Reporte final (E13): cap. 7	Cumple

ID	Requisito	Evidencia	Estado
CAL-05	Aseguramiento de calidad	E05, E08, E16b (main en verde)	En progreso (se confirma con el tag v1.0.0)
CAL-06	Idoneidad funcional y fiabilidad	Reporte final (E13): cap. 9, E16b	Cumple
CAL-07	Enfoque preventivo	E08a, E08b, E08g, E17a–E17d (PR #42 bloqueado por CC 11, cerrado sin merge)	Cumple
CAL-08	Carácter sistemático	E02, E02b, Reporte final (E13): cap. 11	Cumple
MET-00	Métricas aplicando IA	E10h, E10i	Cumple
MET-P1	Complejidad ciclomática	E07, 19d18b6, PR #28	Cumple
MET-P2	Cobertura	E06, 6ad52ac, PR #29	Cumple
MET-P3	Densidad de defectos	E11b, E11c, E11f, E10h	Cumple
MET-R1	MTTD	E11f, E10h	Cumple
MET-R2	MTTR	E11f, E10h	Cumple
MET-R3	Eficacia de pruebas	E11f	En progreso (0 críticos)
MET-J1	Eficacia de revisión	E11f, E15, E10g	Cumple
MET-J2	Desviación tiempo/esfuerzo por módulo	E11d	Cumple
MET-J3	Plazos (+ satisfacción opcional)	E11d	Cumple
EST-1	Juicio de expertos	E12	Cumple
EST-2	Estimación análoga	E12	Cumple
EST-3	Tres puntos (PERT)	E12	Cumple
EST-4	Puntos de función	E12	Cumple
PRO-01	Kanban	E02	En progreso
PRO-02	Commits por cambio	E00c, E02b, E15c; tabla de PRs (cap. 8.2)	Cumple
PRO-03	Evidencia por sprint	E00–E17d	En progreso (falta E14: captura de la matriz final)
PRO-04	Bitácora de IA	docs/AI_LOG.md	Cumple

4.4 Lectura del resultado

Los requisitos del **programa**, **las métricas**, **la estimación** y **el documento** están cumplidos con evidencia. Los que siguen abiertos se explican así:

- **PRG-09** (Revisión en clase): En progreso (guion listo; ensayo pendiente). Criterio: Demo ensayada < 5 min.
- **CAL-05** (Aseguramiento de calidad): En progreso (se confirma con el tag v1.0.0). Criterio: Pipeline verde en v1.0.
- **MET-R3** (Eficacia de pruebas): En progreso (0 críticos). Criterio: Con defectos críticos reales: aún 0 críticos (todas las severidades: $3/3 = 100\%$ antes de producción).
- **PRO-01** (Kanban): En progreso. Criterio: Capturas por sprint.
- **PRO-03** (Evidencia por sprint): En progreso (falta E14: captura de la matriz final). Criterio: E01–E14 con archivo y fecha.

MET-R3 (eficacia de las pruebas sobre errores críticos) se mantiene en progreso a propósito: el historial no tiene ningún defecto crítico y no voy a inventar uno. La fórmula está implementada y probada con datos sintéticos; con datos reales reporta «sin datos» de forma explícita, como exige la regla de evidencia.

5. INFORME DE CALIDAD EN UN FORMATO

En resumen: El programa genera en cada ejecución (local y en CI) un informe de calidad con **formato fijo de 12 secciones**, basado en el informe de finalización de pruebas de ISO/IEC/IEEE 29119-3:2021. Termina en un **dictamen de liberación** con tres valores posibles (LIBERAR, LIBERAR CON CONDICIONES, NO LIBERAR) decidido por reglas explícitas. En GitHub se publica en el resumen de cada ejecución.

5.1 Estructura del informe

`qualityops/report.py` escribe `reports/quality_report.md`. Las secciones y su origen están fijos en el código; por eso dos informes de fechas distintas se pueden comparar línea por línea.

Tabla 11. Las 12 secciones del informe de calidad

#	Sección	Contenido	Fuente del dato
1	Identificación	Proyecto, commit analizado, fecha, versión de la herramienta, fuente de la interpretación e intento de LLM	metrics.json, ai_analysis.json
2	Resumen y dictamen	Dictamen de liberación y recomendaciones	gate.json + reglas
3	Resultados de las pruebas	Total, aprobadas, fallidas, errores, omitidas	pytest (JUnit XML)
4	Criterios de salida	Cada criterio del quality gate con valor, umbral y si cumple	gate.json
5	Métricas de producto	Complejidad, cobertura, tamaño, densidad	metrics.json
6	Métricas de proceso	MTTD, MTTR, eficacia de pruebas, con n	metrics.json
7	Métricas de proyecto	Eficacia de revisión, desviación, plazos	metrics.json
8	Estimación	Las 4 técnicas y las horas reales	metrics.json
9	Defectos por severidad y fase	Matriz severidad × fase de detección	Minería SZZ
10	Interpretación	Valoración por reglas de cada métrica + texto del LLM	ai_advisor.py
11	Verificaciones de coherencia	5 comprobaciones aritméticas entre métricas	ai_advisor.py
12	Aprobación	Responsable y fecha (firma)	Manual

Frente a ISO/IEC/IEEE 29119-3, el informe conserva los elementos del *test completion report*: identificación, resumen, desviaciones respecto a lo planeado (criterios de salida), métricas de la prueba, riesgos residuales (interpretación y recomendaciones) y aprobación.

5.2 Reglas del dictamen de liberación

El dictamen no lo decide la IA. Lo calcula la función `dictamen()` de `report.py` con tres reglas:

Tabla 12. Criterios del dictamen

Dictamen	Condición	Significado
NO LIBERAR	El quality gate no aprobó (o no existe gate.json)	El cambio no debe integrarse
LIBERAR CON CONDICIONES	Gate aprobado, pero alguna métrica está «sin datos» o falla una verificación de coherencia	Puede integrarse; hay huecos que se deben atender

Dictamen	Condición	Significado
LIBERAR	Gate aprobado, todas las métricas con datos y 5/5 verificaciones correctas	Puede integrarse sin reservas

En la ejecución analizada el dictamen es **LIBERAR CON CONDICIONES** porque la métrica «Eficacia de las pruebas» no tiene datos: no hay defectos críticos en el historial, de modo que la eficacia sobre errores críticos no puede calcularse. El programa prefiere declarar el hueco a mostrar un 100 % engañoso.

5.3 Informe real generado en el pipeline



Informe de calidad — QualityOps AI

Formato basado en el informe de finalización de pruebas (ISO/IEC/IEEE 29119-3:2021).

1. Identificación

Campo	Valor
Proyecto	qualityops-ai
Commit analizado	436e614
Fecha de generación	2026-10-08T09:52:58+00:00
Herramienta	QualityOps AI 0.1.0
Fuente de la interpretación	llm
Intento de LLM	usado: la interpretación la redactó el modelo de lenguaje

2. Resumen y dictamen de liberación

Dictamen: **LIBERAR CON CONDICIONES**

- Eficacia de las pruebas: mantener la convención `fix:` con trailers para que el historial genere datos.
- Revisar desviación de tiempo y esfuerzo: -67.6 %: se usaron 4.05 h reales contra 12.5 h estimadas en 16 tareas (menos esfuerzo del previsto).
- Revisar cumplimiento de plazos: 37.5 % de tareas cerradas en la fecha de su sprint; tarde: T02, T08, T09, T10, T11, T12, T13, T14, T15, T17.

Figura 12. Informe de calidad publicado en el resumen de GitHub Actions (PR #40) (evidencia E10k)

Qué muestra. El resumen del job «calidad» en verde. La sección 1 (Identificación) indica el commit analizado (436e614), la fecha, la versión de la herramienta, la fuente de la interpretación (llm) y la fila «Intento de LLM: usado». La sección 2 muestra el dictamen LIBERAR CON CONDICIONES y sus tres recomendaciones.

Qué demuestra. El informe con formato se genera automáticamente en CI en cada Pull Request, con la IA generativa funcionando en el servidor de GitHub (la llave se lee del secreto `GEMINI_API_KEY`).

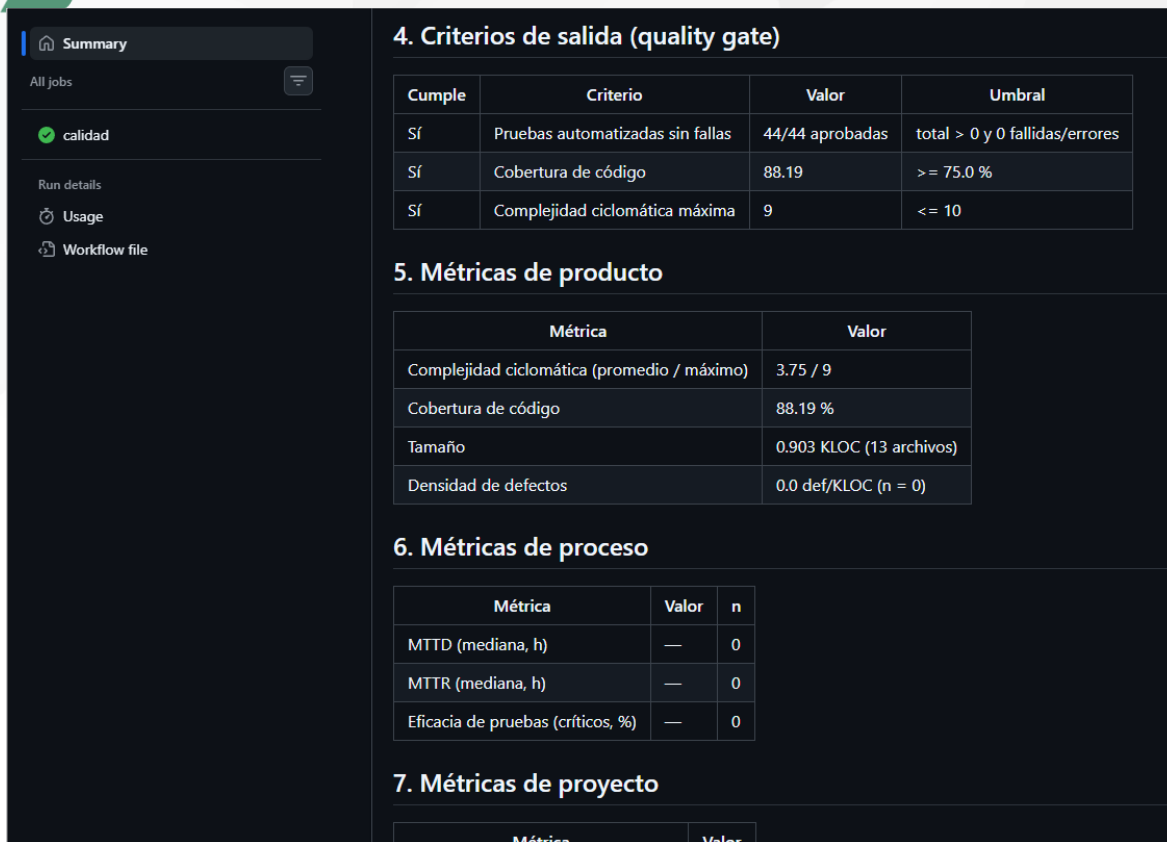


Figura 13. Secciones 4 a 6 del informe en Actions (PR #38) (evidencia E10_actions_PR38_summary_3)

Qué muestra. Los criterios de salida con «Sí» en los tres, la tabla de métricas de producto y la de proceso. En esa ejecución aún no había defectos, por lo que MTTD y MTTR aparecen como «—» con n = 0.

Qué demuestra. El informe no rellena huecos: cuando una métrica no tiene datos lo dice explícitamente (n = 0).

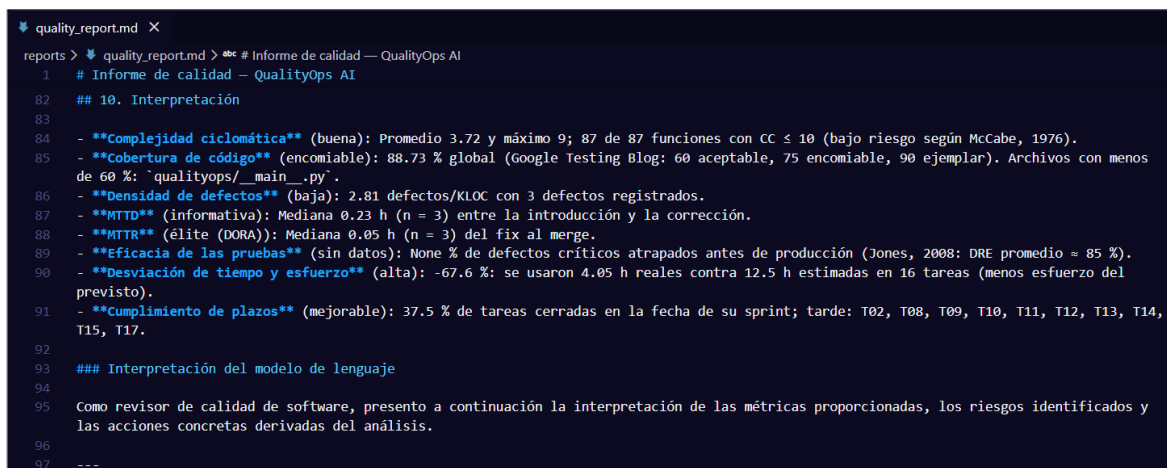


Figura 14. Sección 10 del informe: interpretación por reglas y del modelo de lenguaje (evidencia E10i)

Qué muestra. La interpretación por reglas de cada métrica con su valoración (buena, encomiable, baja, élite DORA, sin datos, alta, mejorable) y su referencia, seguida del bloque «Interpretación del modelo de lenguaje» redactado por Gemini.

Qué demuestra. Las dos rutas de interpretación conviven: la de reglas siempre existe y la del LLM se agrega cuando está disponible.

6. REQUISITOS EXPLÍCITOS

En resumen: Los requisitos explícitos del programa son **15 funcionales** y **10 no funcionales**, cada uno con prioridad, módulo responsable, criterio de aceptación verificable y evidencia. Este capítulo los especifica módulo por módulo con su entrada, su proceso y su salida, y muestra la captura real que prueba que el requisito se cumple.

6.1 Catálogo de requisitos funcionales

Un requisito funcional describe **qué hace** el programa. Todos derivan de la actividad (métricas de producto, proceso y proyecto; estimación; DevOps; IA) y cada uno tiene un criterio que se comprueba con una salida real.

Tabla 13. Requisitos funcionales

ID	Requisito	Prioridad	Módulo	Criterio de aceptación	Evidencia
RF-01	Calcular la complejidad ciclomática de cada función, con promedio, mediana, máximo y conteo de funciones con $CC \leq 10$	Alta	product_metrics.py	Coincide función por función con <code>radon cc -s</code>	E07
RF-02	Contar LOC, SLOC, comentarios, blancos y KLOC del código de producto (sin pruebas)	Alta	product_metrics.py	Coincide con <code>radon raw -s</code>	E11c
RF-03	Ejecutar las pruebas del repositorio y medir la cobertura de líneas global y por archivo	Alta	coverage_metrics.py	Coincide con <code>coverage report</code>	E06
RF-04	Identificar los defectos reales del historial de Git (commits <code>fix:</code>), su commit inductor (SZZ), severidad y fase	Alta	defects.py	Defectos de los trailers = defectos minados	E11f
RF-05	Calcular la densidad de defectos global y por archivo (defectos/KLOC)	Alta	density.py	Densidad = defectos / KLOC (verificación 3)	E11b
RF-06	Calcular MTMD, MTTR y eficacia de las pruebas (críticos y todas las severidades)	Alta	process_metrics.py	Valores con n explícito; «sin datos» si n = 0	E11f
RF-07	Calcular eficacia de la revisión, desviación de tiempo/esfuerzo por módulo y cumplimiento de plazos	Alta	project_metrics.py	Coincide con el cálculo manual sobre <code>time_log</code>	E11d
RF-08	Aplicar 4 técnicas de estimación: juicio de expertos, análoga, tres puntos y puntos de función	Alta	estimation.py	Las 4 técnicas sobre la misma línea base	E12
RF-09	Ofrecer una CLI única que analice cualquier repositorio (<code>--repo</code>) y guarde <code>metrics.json</code>	Alta	cli.py	JSON completo y resumen en terminal	E04b, E10h
RF-10	Aplicar un quality gate con umbrales configurables y código de salida 0/1	Alta	quality_gate.py	Bloquea con umbral de demostración; aprueba con el real	E08a
RF-11	Ejecutar el análisis completo en GitHub Actions en cada push y Pull Request a main	Alta	ci.yml	Job «calidad» visible y obligatorio	E08, E08b
RF-12	Generar el informe de calidad de 12 secciones con dictamen de liberación	Alta	report.py	12 secciones en local y en CI	E10, E10k
RF-13	Verificar la coherencia de las métricas e interpretarlas con LLM, con respaldo por reglas	Alta	ai_advisor.py	fuelle = llm cuando hay llave; reglas si falla	E10h, E10m
RF-14	Mostrar las métricas en un dashboard web	Media	app.py, dashboard.py	6 pestañas con datos de <code>metrics.json</code>	E09a–h
RF-15	Calcular el grado de cumplimiento desde la matriz de requisitos	Media	dashboard.py	% = Cumple / total	E09b

6.2 Catálogo de requisitos no funcionales

Un requisito no funcional describe **cómo** debe comportarse el programa. Los más importantes (RNF-02 y RNF-03) no dependen de la disciplina del autor: los hace cumplir el quality gate en cada Pull Request.

Tabla 14. Requisitos no funcionales

ID	Atributo	Requisito	Cómo se verifica
RNF-01	Reproducibilidad	Dependencias con versión fija; Python 3.14 en local y en CI	requirements.txt, E00b
RNF-02	Mantenibilidad	Ninguna función con complejidad ciclomática > 10	Gate: CC máx. ≤ 10
RNF-03	Verificabilidad	Cobertura de líneas ≥ 75 % y 0 pruebas fallidas	Gate
RNF-04	Tolerancia a fallos	Si el LLM no responde o no hay llave, el informe se genera por reglas	E10m, E10g, pruebas
RNF-05	Seguridad	La llave de API vive como secreto; al LLM solo se envían métricas agregadas	ci.yml, D19
RNF-06	Portabilidad	Analiza cualquier repositorio Python con Git mediante <code>--repo</code>	D13, pruebas con repos sintéticos
RNF-07	Eficiencia	El pipeline completo termina en menos de 1 minuto	E08 (34 s), E10 (45 s)
RNF-08	Trazabilidad	Conventional Commits + trailers en cada <code>fix:</code>	E15c
RNF-09	Usabilidad	Salidas e informe en español, con valores, umbrales y explicación	E10k, E09
RNF-10	Integridad de datos	5 verificaciones de coherencia entre métricas en cada ejecución	Sección 11 del informe

6.3 Especificación por módulo

Cada apartado especifica un módulo con una tabla de **entradas, proceso y salidas** y presenta la evidencia de su funcionamiento. El orden sigue el flujo de los datos: primero se mide el producto, luego el historial, después se valida y por último se presenta.

6.3.1 Complejidad ciclomática y tamaño (RF-01, RF-02) — `product_metrics.py`

La complejidad ciclomática (McCabe, 1976) cuenta los caminos independientes de una función: $1 + \text{número de decisiones}$ (`if`, `for`, `while`, `and`, `or`, `except...`). A más caminos, más difícil es probar y mantener la función. El módulo usa `radon` para calcularla por función y para contar las líneas.

Tabla 15. Entradas, proceso y salidas de `product_metrics.py`

Elemento	Descripción
Entradas	Ruta del repositorio (<code>--repo</code>). Se recorren los <code>*.py</code> excluyendo <code>.env</code> , <code>tests</code> , cachés y archivos <code>test_*.py</code> .
Proceso	<code>cc_visit</code> de <code>radon</code> calcula la CC de cada función y método; <code>analyze</code> cuenta LOC, SLOC, comentarios y blancos.
Salidas	Lista de funciones (archivo, nombre, línea, CC, rango A–F) y estadísticas: n, promedio, mediana, máximo y funciones con $CC \leq 10$.
Reglas y validaciones	Si no hay código, los valores son <code>null</code> (no se inventa un 0). Las clases no se cuentan dos veces.


```
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> python -m qualityops.product_metrics --repo .

Complejidad ciclomática - C:\Users\LPC\proyectos\qualityops-ai
CC Rango Función (archivo:línea)
6 B analizar_complejidad (qualityops/product_metrics.py:68)
6 B contar_lineas (qualityops/product_metrics.py:117)
5 A encontrar_archivos_python (qualityops/product_metrics.py:45)
4 A _estadisticas (qualityops/product_metrics.py:100)
3 A es_archivo_de_prueba (qualityops/product_metrics.py:39)
2 A _imprimir (qualityops/product_metrics.py:149)
1 A _leer (qualityops/product_metrics.py:63)

Funciones: 7 · promedio: 3.86 · mediana: 4 · máximo: 6 · CC <= 10: 7
Tamaño: 2 archivos · LOC 175 · SLOC 96 · KLOC 0.096 · comentarios 7 · blancos 34

(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> radon cc -s qualityops
qualityops\product_metrics.py
F 68:0 analizar_complejidad - B (6)
F 117:0 contar_lineas - B (6)
F 45:0 encontrar_archivos_python - A (5)
F 100:0 _estadisticas - A (4)
F 39:0 es_archivo_de_prueba - A (3)
F 149:0 _imprimir - A (2)
F 63:0 _leer - A (1)
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> |
```

Figura 15. Complejidad por función calculada por QualityOps comparada con radon (evidencia E07)

Qué muestra. Arriba, la salida de `python -m qualityops.product_metrics --repo .` con las 7 funciones que había en T04, su CC y su rango. Abajo, `radon cc -s qualityops`, la herramienta de referencia.

Qué demuestra. Los 7 valores coinciden función por función (6, 6, 5, 4, 3, 2, 1): el cálculo es correcto (criterio de RF-01). La primera versión generada por la IA tenía una función con CC 11; se dividió antes de integrarla.

```
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> radon raw qualityops -s
qualityops\coverage_metrics.py
LOC: 146
LLOC: 82
SLOC: 91
Comments: 7
Single comments: 10
Multi: 19
Blank: 26
- Comment Stats
  (C % L): 5%
  (C % S): 8%
  (C + M % L): 18%
qualityops\defects.py
LOC: 252
LLOC: 150
SLOC: 163
Comments: 12
Single comments: 22
Multi: 25
Blank: 42
- Comment Stats
  (C % L): 5%
  (C % S): 7%
  (C + M % L): 15%
qualityops\density.py
LOC: 74
LLOC: 35
SLOC: 37
Comments: 0
Single comments: 3
Multi: 16
Blank: 18
- Comment Stats
  (C % L): 0%
  (C % S): 0%
  (C + M % L): 22%
qualityops\product_metrics.py
LOC: 168
LLOC: 77
SLOC: 95
Comments: 7
```

Figura 16. Conteo de líneas de radon usado para validar el SLOC (evidencia E11c)

Qué muestra. `radon raw -s` por archivo: LOC, LLOC, SLOC, comentarios y blancos.

Qué demuestra. El SLOC de cada archivo (91, 163, 37, 95) es el mismo que reporta QualityOps en la densidad de defectos (RF-02).

6.3.2 Pruebas y cobertura (RF-03) — `coverage_metrics.py`

La cobertura de código es el porcentaje de líneas ejecutables que las pruebas automatizadas ejecutan. Una cobertura baja indica código que nunca se prueba.

Tabla 16. Entradas, proceso y salidas de `coverage_metrics.py`

Elemento	Descripción
Entradas	Ruta del repositorio.
Proceso	Ejecuta <code>python -m pytest --cov=. --cov-report=json --junitxml=...</code> en un proceso aparte y en una carpeta temporal, y lee ambos reportes.
Salidas	Pruebas (total, aprobadas, fallidas, errores, omitidas, código de salida) y cobertura global y por archivo.
Reglas y validaciones	Solo cuenta archivos de producto (los mismos de RF-01). Si pytest falla, se reportan las fallas en lugar de detenerse.

```
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> python -m qualityops.coverage_metrics --repo .

Pruebas - C:\Users\LPC\proyectos\qualityops-ai
Total 8 · aprobadas 8 · fallidas 0 · errores 0 · omitidas 0 · código de salida 0

Ejecutables Ejecutadas      % Archivo
1          1      100.0 qualityops/__init__.py
67         54      80.6 qualityops/coverage_metrics.py
61         49      80.33 qualityops/product_metrics.py

Cobertura global del código de producto: 80.62 % (104 de 129 líneas)

(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> python -m pytest -q --cov=qualityops --cov-report=term
[100%]
===== tests coverage =====
----- coverage: platform win32, python 3.14.3-final-0 -----

Name                               Stmts  Miss  Cover
-----
qualityops\__init__.py              1     0   100%
qualityops\coverage_metrics.py      67    13    81%
qualityops\product_metrics.py       61    12    80%
-----
TOTAL                               129    25    81%
8 passed in 5.88s
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> |
```

Figura 17. Cobertura de QualityOps comparada con coverage report (evidencia E06)

Qué muestra. Arriba, `python -m qualityops.coverage_metrics --repo .` (8 pruebas aprobadas; 80.62 % global, 104 de 129 líneas). Abajo, `pytest --cov --cov-report=term`, la herramienta estándar (81 % redondeado).

Qué demuestra. Las líneas ejecutables y ejecutadas coinciden por archivo (1/1, 67/54, 61/49); el porcentaje de QualityOps solo tiene más decimales (criterio de RF-03).

6.3.3 Minería de defectos SZZ (RF-04) — defects.py

El algoritmo SZZ (Śliwerski, Zimmermann y Zeller, 2005) encuentra el commit que **introdujo** un defecto a partir del commit que lo **corrigió**: toma las líneas que modificó la corrección y, con `git blame` sobre la versión anterior, identifica el último commit que las tocó. Adapté la minería de mi herramienta anterior `metricas` a Python y a los trailers de este proyecto (D07, D08).

Tabla 17. Entradas, proceso y salidas de defects.py

Elemento	Descripción
Entradas	Historial de Git del repositorio (commits, merges con <code>--first-parent</code> , tags vX.Y.Z).
Proceso	Para cada commit <code>fix</code> : lee los trailers; si cambia código de producto, ejecuta <code>git blame --porcelain</code> sobre el padre para hallar el inductor; ubica el merge que lo integró y el tag que lo liberó.
Salidas	Por defecto: <code>fix</code> , inductor, fechas, MTTD (h), merge, MTTR (h), severidad, fase y fuente de la fase. También la lista de fixes excluidos.
Reglas y validaciones	Fase por trailer <code>Detectado-en</code> ; si falta, se deduce (antes del merge = revisión; después de un tag = producción). Los fixes que solo tocan pruebas o documentación se excluyen con su motivo.

```
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> python -m qualityops.defects --repo .

Minería de defectos (SZZ) - commit 78be41c
Commits sin merge: 28 · merges de PR: 13 · tags de versión: ninguno
Defectos de producto: 3 · fixes excluidos: 0

a02edd4 menor revision (trailer) MTTD 0.25 h · MTTR 0.03 h · inductor c8b2162
fix(report): el informe y el dashboard explican si el LLM se usó, falló o no está configurado
f82e42f mayor pruebas (trailer) MTTD 0.23 h · MTTR 0.05 h · inductor c8b2162
fix(ai_advisor): elimina el proveedor GitHub Models, retirado por GitHub el 2026-07-30
a6139f9 menor revision (trailer) MTTD 0.22 h · MTTR 0.13 h · inductor cf916cb
fix(ai_advisor): nombres de archivo con guion bajo se mostraban en negritas en el informe

MTTD mediana: 0.23 h (n = 3)
```

Figura 18. Minería de defectos sobre main: tres defectos reales (evidencia E11f)

Qué muestra. `python -m qualityops.defects --repo .` sobre el commit 78be41c: 28 commits, 13 merges, sin tags y 3 defectos de producto. Para cada uno: hash del fix, severidad, fase (leída del trailer), MTTD, MTTR e inductor.

Qué demuestra. Los defectos no se capturaron a mano: salen del historial. Por ejemplo, f82e42f es el defecto 2 (mayor, detectado en pruebas) con MTTD 0.23 h y MTTR 0.05 h, inducido por c8b2162.

6.3.4 Densidad de defectos (RF-05) — density.py

Tabla 18. Entradas, proceso y salidas de density.py

Elemento	Descripción
Entradas	Defectos minados (RF-04) y SLOC por archivo (RF-02).
Proceso	Asigna cada defecto a los archivos de producto que tocó su corrección y divide entre los KLOC de ese archivo; el global usa defectos únicos entre los KLOC totales.
Salidas	Densidad global y por archivo (defectos/KLOC).
Reglas y validaciones	Con código y sin defectos devuelve 0.0 (dato real); sin código devuelve <code>null</code> .

```
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> python -m qualityops.density --repo .

Densidad de defectos - C:\Users\LPC\proyectos\qualityops-ai
Defectos SLOC Def/KLOC Archivo
0 1 0.0 qualityops/__init__.py
0 91 0.0 qualityops/coverage_metrics.py
0 163 0.0 qualityops/defects.py
0 37 0.0 qualityops/density.py
0 95 0.0 qualityops/product_metrics.py

Global: 0 defectos / 0.387 KLOC = 0.0 defectos/KLOC
```

Figura 19. Densidad de defectos por archivo (T07, antes del primer defecto) (evidencia E11b)

Qué muestra. La densidad cuando aún no había defectos: 0 defectos / 0.387 KLOC = 0.0.

Qué demuestra. La fórmula y el desglose por archivo funcionan; el valor actual (2.81 def/KLOC) se analiza en el capítulo 12.

6.3.5 Métricas de proceso y de proyecto (RF-06, RF-07)

Tabla 19. Entradas, proceso y salidas de process_metrics.py y project_metrics.py

Elemento	Descripción
Entradas	Defectos minados, data/time_log.csv y el calendario de sprints de pyproject.toml.
Proceso	MTTD = fecha fix – fecha inductor; MTTR = fecha merge – fecha fix; eficacias = defectos en la fase / defectos con fase conocida; desviación = (real – estimado) / estimado por módulo; plazos = fecha de cierre ≤ fecha del sprint.
Salidas	Promedio y mediana con n; porcentajes con numerador y denominador; desviación por módulo y total; lista de tareas tarde.
Reglas y validaciones	Solo se usan tareas terminadas; con n = 0 el valor es null y se reporta «sin datos».

```

create mode 100644 evidence/E00c_gh_pr_checks_T11_2020-10-08_0010.png
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> python -m pytest -q
[100%]
32 passed in 24.82s
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> ruff check .
All checks passed!
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> python -m qualityops --repo . --salida reports
QualityOps AI 0.1.0 – qualityops-ai @ 415d819
Pruebas:      32/32 aprobadas · 0 fallidas · 0 errores
Cobertura:    85.68 %
Complejidad:  promedio 3.47 · máximo 9
Tamaño:       0.615 KLOC en 10 archivos
Defectos:     0 · densidad 0.0 def/KLOC
MTTD / MTTR:  None h / None h (n = 0 / 0)
Desviación:   -52.8 % (11 tareas terminadas)
Plazos:       54.55 % a tiempo

Resultados guardados en reports\metrics.json
  
```

Figura 20. Primera ejecución con métricas de proceso y proyecto (T13–T14) (evidencia E11d)

Qué muestra. 32/32 pruebas, cobertura 85.68 %, MTTD y MTTR **None** con n = 0 (aún no había defectos), desviación –52.8 % sobre 11 tareas terminadas y 54.55 % de tareas a tiempo.

Qué demuestra. Con n = 0 el programa no inventa valores, y la desviación y los plazos ya se calculaban con datos reales del registro de tiempos.

6.3.6 Estimación (RF-08) — estimation.py

Tabla 20. Entradas, proceso y salidas de estimation.py

Elemento	Descripción
Entradas	Tareas de la línea base (time_log.csv) y data/estimacion.json (conteo de PF y proyecto de referencia).
Proceso	Juicio = Σ más probable; PERT = $\Sigma (O + 4M + P)/6$ con σ combinada; PF = Σ cantidad × peso IFPUG; horas PF = PF × tasa; análoga = horas de la referencia × (PF propios / PF referencia) × factor.
Salidas	Horas por técnica, σ y rango del 95 % para PERT, PF sin ajustar, tasa y su origen.
Reglas y validaciones	Las tareas agregadas después de la línea base se excluyen; si falta un dato, esa técnica no se calcula.

```

create mode 100644 Evidence/E11d_cli_proceso_proyecto_713714_2020-10-08_0050.png
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> python -m pytest -q
.....[100%]
44 passed in 25.06s
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> ruff check .
All checks passed!
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> python -m qualityops --repo . --salida reports
QualityOps AI 0.1.0 - qualityops-ai @ 6de8c75
Pruebas: 44/44 aprobadas · 0 fallidas · 0 errores
Cobertura: 88.19 %
Complejidad: promedio 3.75 · máximo 9
Tamaño: 0.903 KLOC en 13 archivos
Defectos: 0 · densidad 0.0 def/KLOC
MTTD / MTTR: None h / None h (n = 0 / 0)
Desviación: -60.0 % (13 tareas terminadas)
Plazos: 46.15 % a tiempo
Estimación: juicio 22.5 h · PERT 24.79 h · PF 4.24 h · análoga 4.79 h

Resultados guardados en reports\metrics.json
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> python -m qualityops.quality_gate

Quality gate - QualityOps AI
[OK ] Pruebas automatizadas sin fallas: 44/44 aprobadas (umbral total > 0 y 0 fallidas/errores)
[OK ] Cobertura de código: 88.19 (umbral >= 75.0 %)
[OK ] Complejidad ciclomática máxima: 9 (umbral <= 10)

Dictamen: APROBADO

(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> python -m qualityops.report
Informe generado: reports\quality_report.md · dictamen: LIBERAR CON CONDICIONES · interpretación: reglas

```

Figura 21. CLI con las cuatro técnicas de estimación (evidencia E12)

Qué muestra. La salida con la nueva línea «Estimación: juicio 22.5 h · PERT 24.79 h · PF 4.24 h · análoga 4.79 h», además del quality gate aprobado y el informe generado.

Qué demuestra. Las cuatro técnicas se calculan con el programa, no en una hoja aparte (capítulo 13).

6.3.7 CLI y archivo de métricas (RF-09) — cli.py

Tabla 21. Entradas, proceso y salidas de cli.py

Elemento	Descripción
Entradas	--repo (repositorio a analizar) y --salida (carpeta de reportes).
Proceso	Llama a todos los módulos anteriores, agrega herramienta, fecha, repositorio y commit, y guarda el JSON.
Salidas	metrics.json con las claves herramienta, generado, repo, commit, producto, pruebas, proceso, proyecto, estimacion y defectos; resumen en terminal.
Reglas y validaciones	El commit analizado se registra para que cada informe sea trazable a una versión del código.

```
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> python -m qualityops --repo . --salida reports
QualityOps AI 0.1.0 - qualityops-ai @ 1b1c11e
Pruebas: 17/17 aprobadas · 0 fallidas · 0 errores
Cobertura: 81.88 %
Complejidad: promedio 3.3 · máximo 9
Tamaño: 0.449 KLOC en 7 archivos
Defectos: 0 · densidad 0.0 def/KLOC

Resultados guardados en reports\metrics.json
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai>
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai>
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> Get-Content reports\metrics.json -TotalCount 40
{
  "herramienta": {
    "nombre": "QualityOps AI",
    "version": "0.1.0"
  },
  "generado": "2026-10-06T12:22:43-06:00",
  "repo": "qualityops-ai",
  "commit": "1b1c11e",
  "producto": {
    "complejidad": {
      "funciones": [
        {
          "archivo": "qualityops/cli.py",
          "nombre": "commit_actual",
          "linea": 26,
          "cc": 2,
          "rango": "A"
        },
        {
          "archivo": "qualityops/cli.py",
          "nombre": "recolectar_metricas",
          "linea": 33,
          "cc": 1,
          "rango": "A"
        },
        {
          "archivo": "qualityops/cli.py",
          "nombre": "guardar",
          "linea": 55,
          "cc": 1,
          "rango": "A"
        },
        {
          "archivo": "qualityops/cli.py",
          "nombre": "resumen",
          "linea": 63,
          "cc": 1,
          "rango": "A"
        }
      ]
    }
  }
}
```

Figura 22. Primera ejecución de la CLI y estructura de metrics.json (T08) (evidencia E04b)

Qué muestra. El resumen en terminal (17/17 pruebas, cobertura 81.88 %, CC máx. 9, 0.449 KLOC en 7 archivos) y las primeras 40 líneas de `metrics.json`, con la complejidad de cada función de `cli.py`.

Qué demuestra. La salida es un contrato estructurado que pueden leer el gate, el informe y el dashboard.

6.3.8 Quality gate (RF-10) — quality_gate.py

Tabla 22. Entradas, proceso y salidas de quality_gate.py

Elemento	Descripción
Entradas	<code>metrics.json</code> y los umbrales de <code>[tool.qualityops.umbrales]</code> en <code>pyproject.toml</code> (o <code>--config</code>).
Proceso	Evalúa tres criterios: pruebas sin fallas (total > 0, 0 fallidas y 0 errores), cobertura ≥ 75.0 % y CC máxima ≤ 10 .
Salidas	Tabla en terminal, <code>gate.json</code> y, en CI, una tabla Markdown en el resumen de Actions. Código de salida 0 (aprobado) o 1 (bloqueado).
Reglas y validaciones	Umbrales con referencia: 75 % «encomiable» (Google Testing Blog) y CC ≤ 10 (McCabe). Un valor faltante cuenta como falla.

```
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> Set-Content "$env:TEMP\umbral_demo.toml" "[tool.qualityops.umbrales]'ncobertura_minima = 95.0'"
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> python -m qualityops.quality_gate --config "$env:TEMP\umbral_demo.toml"; echo "codigo de salida: $LASTEXITCODE"

Quality gate - QualityOps AI
[OK ] Pruebas automatizadas sin fallas: 24/24 aprobadas (umbral total > 0 y 0 fallidas/errores)
[FALLA] Cobertura de código: 83.52 (umbral >= 95.0 %)
[OK ] Complejidad ciclomática máxima: 9 (umbral <= 10)

Dictamen: BLOQUEADO

codigo de salida: 1
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> python -m qualityops.quality_gate; echo "codigo de salida: $LASTEXITCODE"

Quality gate - QualityOps AI
[OK ] Pruebas automatizadas sin fallas: 24/24 aprobadas (umbral total > 0 y 0 fallidas/errores)
[OK ] Cobertura de código: 83.52 (umbral >= 75.0 %)
[OK ] Complejidad ciclomática máxima: 9 (umbral <= 10)

Dictamen: APROBADO

codigo de salida: 0
```

Figura 23. Quality gate bloqueando con un umbral de demostración y aprobando con el real (evidencia E08a_gate)

Qué muestra. Primero creé un archivo de umbrales de demostración con cobertura mínima de 95 %: el gate marca [FALLA] en cobertura (83.52 %), dictamen BLOQUEADO y código de salida 1. Con los umbrales reales (75 %) los tres criterios están [OK], dictamen APROBADO y código 0.

Qué demuestra. El gate realmente detiene el flujo: el código 1 es lo que hace que GitHub Actions marque el check en rojo (RF-10, enfoque preventivo).

6.3.9 Pipeline de integración continua (RF-11) — ci.yml

Tabla 23. Entradas, proceso y salidas de ci.yml

Elemento	Descripción
Entradas	Evento push o pull_request hacia main.
Proceso	Siete pasos (ver la figura del pipeline del capítulo 3).
Salidas	Check «calidad» verde o rojo, resumen con el gate y el informe, y artefacto <code>reportes-calidad</code> .
Reglas y validaciones	El ruleset <code>proteger-main</code> exige el check para hacer merge; <code>permissions: contents: read</code> limita al mínimo los permisos del token.

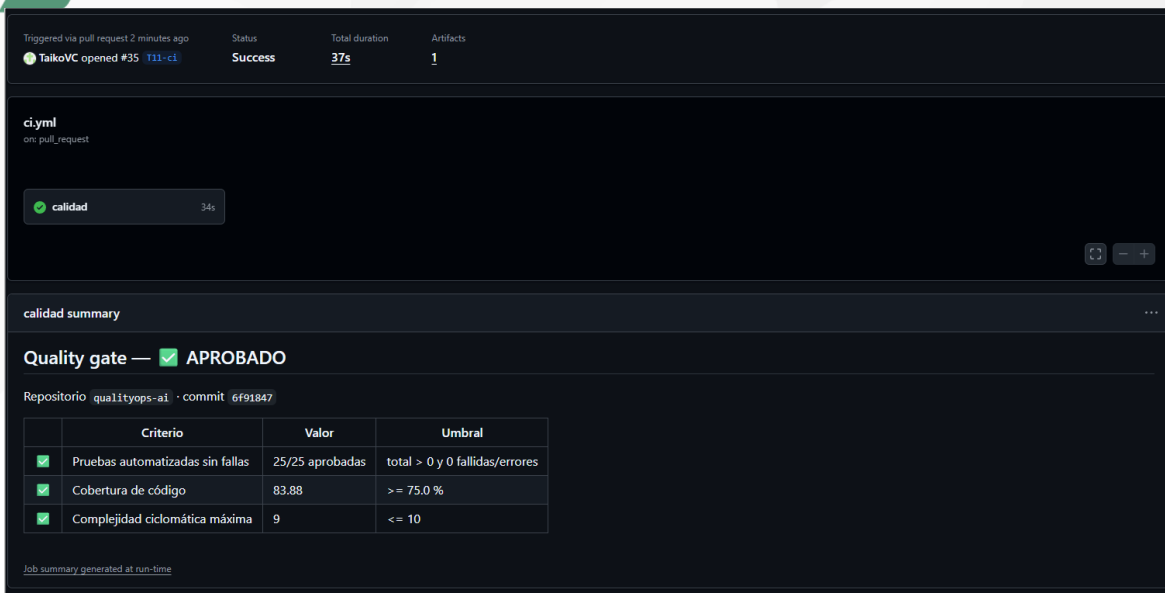


Figura 24. Primera ejecución del pipeline: job «calidad» y resumen del quality gate (PR #35) (evidencia E08_actions)

Qué muestra. El run disparado por el PR #35 (T11-ci): estado *Success*, duración total 37 s, 1 artefacto, y el resumen «Quality gate — APROBADO» con 25/25 pruebas, cobertura 83.88 % y CC máxima 9 para el commit 6f91847.

Qué demuestra. El pipeline funcionó en verde desde su primera ejecución y publica el resultado donde lo ve cualquier revisor del PR (RF-11, RNF-07).

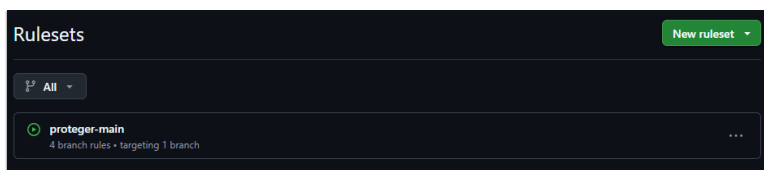


Figura 25. Ruleset proteger-main activo (evidencia E08b)

Qué muestra. La configuración del repositorio con el ruleset `proteger-main` activo, con 4 reglas sobre 1 rama.

Qué demuestra. Las reglas (PR obligatorio, check «calidad» obligatorio, bloqueo de force push) convierten el gate en una barrera real y no en un aviso.

6.3.10 Informe de calidad (RF-12) — report.py

Tabla 24. Entradas, proceso y salidas de report.py

Elemento	Descripción
Entradas	<code>metrics.json</code> , <code>gate.json</code> y el análisis del AI Advisor.
Proceso	Arma las 12 secciones con formato Markdown y aplica las reglas del dictamen.
Salidas	<code>quality_report.md</code> y <code>ai_analysis.json</code> ; en CI, el informe se agrega al resumen del run.
Reglas y validaciones	La fila «Intento de LLM» explica si la IA se usó, falló (con el motivo) o no está configurada.



Figura 26. Informe en Actions después de corregir el defecto 1 (evidencia E15b)

Qué muestra. Las secciones 9 y 10 del informe del PR #38: el defecto 1 aparece en la tabla (menor, revisión) y el nombre `qualityops/_main_.py` ya se muestra como código, no en negritas.

Qué demuestra. El propio informe ayudó a detectar un defecto de formato durante la revisión del PR y mostró su corrección (ver capítulo 10).

6.3.11 AI Advisor (RF-13) — ai_advisor.py

Tabla 25. Entradas, proceso y salidas de ai_advisor.py

Elemento	Descripción
Entradas	<code>metrics.json</code> , <code>gate.json</code> y, opcionalmente, la llave <code>GEMINI_API_KEY</code> (o <code>ANTHROPIC_API_KEY</code>).
Proceso	1) 5 verificaciones de coherencia; 2) interpretación por reglas con referencias; 3) si hay llave, envía un prompt fijo al LLM con las métricas ya calculadas y la instrucción de no recalcularlas.
Salidas	<code>fuentes</code> (llm o reglas), interpretaciones, recomendaciones, verificaciones, texto del LLM, prompt y proveedor, o <code>error_llm</code> .
Reglas y validaciones	429/503 se reintentan 3 veces con espera creciente; un tiempo agotado no se reintenta; cualquier error deja la ruta por reglas.

```

(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> python -m pytest -q
..... [100%]
55 passed in 185.37s (8:01:45)
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> python -m qualityops.report
Informe generado: reports\quality_report.md · dictamen: LIBERAR CON CONDICIONES · interpretación: reglas
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> Get-Content reports\ai_analysis.json -Encoding UTF8 | Select-String '"fuente"|"proveedor"|"error_llm"'
"fuente": "reglas",
"error_llm": "TimeoutError: The read operation timed out"

```

Figura 27. Tolerancia a fallos: el LLM no responde y el informe se genera por reglas (evidencia E10m)

Qué muestra. Con el modelo `gemini-3.8-flash`, la petición agotó el tiempo de espera; el informe se generó igual, con `fuentes: reglas` y el motivo en `error_llm` («TimeoutError: The read operation timed out»).

Qué demuestra. Una falla del servicio externo no rompe el pipeline ni oculta el problema (RNF-04). Con el modelo `gemini-3.5-flash-lite` la llamada funcionó (Figura de ejecución del capítulo 3).

6.3.12 Dashboard y grado de cumplimiento (RF-14, RF-15) — app.py y dashboard.py

Tabla 26. Entradas, proceso y salidas de app.py y dashboard.py

Elemento	Descripción
Entradas	reports/*.json y docs/MATRIZ_CUMPLIMIENTO.md (rutas configurables por variables de entorno).
Proceso	Streamlit dibuja 6 pestañas; dashboard.py contiene la lógica sin interfaz (lectura de reportes, matriz, tablas) para poder probarla.
Salidas	Pestañas: Resumen y cumplimiento, Producto, Proceso, Proyecto, Estimación y AI Advisor.
Reglas y validaciones	El dashboard se prueba con streamlit.testing.AppTest en lugar de excluirlo de la cobertura (D17).

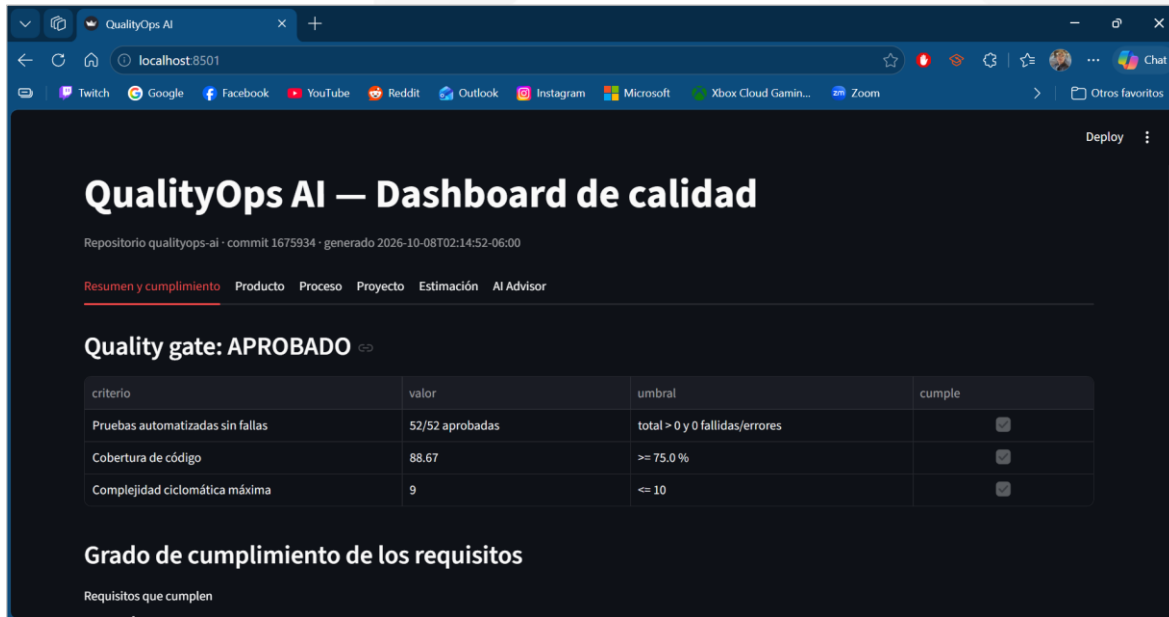


Figura 28. Dashboard: pestaña de resumen con el quality gate (evidencia E09a)

Qué muestra. El dashboard en `localhost:8501` con el repositorio, commit (1675934) y fecha de los datos, las 6 pestañas y la tabla del quality gate APROBADO (52/52 pruebas, cobertura 88.67 %, CC máx. 9) en el momento de la captura.

Qué demuestra. Los mismos datos del JSON se presentan de forma visual para quien no usa la terminal (RF-14).



Figura 29. Dashboard: pestaña de producto (evidencia E09c)

Qué muestra. Cuatro indicadores (cobertura 88.67 %, CC promedio/máxima 3.71/9, tamaño 1.035 KLOC y densidad 0.97 def/KLOC, cuando solo existía el defecto 1) y la gráfica de cobertura por archivo.

Qué demuestra. Cada pestaña corresponde a un grupo de métricas de la actividad; los valores cambian con cada ejecución porque se leen de `metrics.json`.

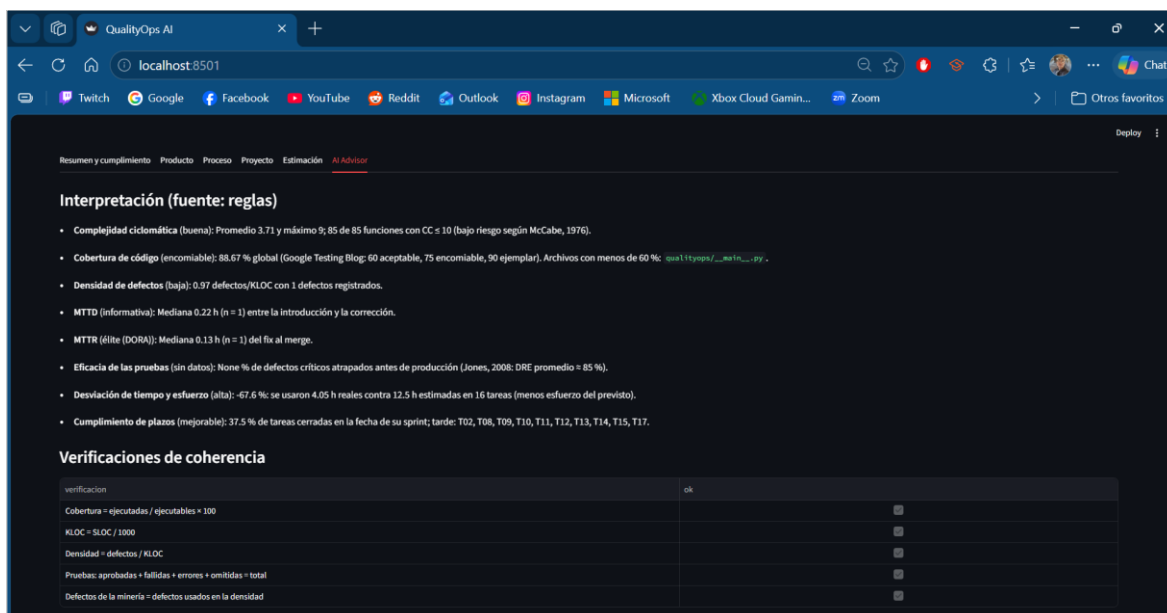


Figura 30. Dashboard: pestaña AI Advisor (evidencia E09h)

Qué muestra. «Interpretación (fuente: reglas)» con la valoración y la referencia de cada métrica, y la tabla de las 5 verificaciones de coherencia, todas marcadas. La captura es anterior a la configuración de Gemini.

Qué demuestra. La interpretación de la IA es visible también fuera del informe.

6.4 Trazabilidad entre requisitos, pruebas y evidencia

Cada requisito funcional está cubierto por al menos un archivo de pruebas automatizadas. La Tabla siguiente se construye con el conteo real de funciones `test_` de cada archivo de `tests/`.

Tabla 27. Requisitos funcionales y pruebas automatizadas que los verifican

Requisitos	Archivo de pruebas	Pruebas
RF-01, RF-02	<code>test_product_metrics.py</code>	5
RF-03	<code>test_coverage_metrics.py</code>	3
RF-04	<code>test_defects.py</code>	5
RF-05	<code>test_density.py</code>	3
RF-06, RF-07	<code>test_process_project_metrics.py</code>	7
RF-08	<code>test_estimation.py</code>	6
RF-09	<code>test_cli.py</code>	1
RF-10	<code>test_quality_gate.py</code>	8
RF-12, RF-13	<code>test_ai_advisor_report.py</code>	13
RF-14, RF-15	<code>test_dashboard.py</code>	5
RF-11	Ejecución real del pipeline (no se prueba con pytest)	—
Total		56

Las pruebas de la minería de defectos crean repositorios Git sintéticos con fechas controladas para comprobar MTDD, MTTR y fases con valores conocidos; las del AI Advisor simulan la API de Gemini (respuesta correcta, error 503 con reintentos y tiempo agotado) para no depender de un servicio externo.

7. REQUISITOS DEL USUARIO Y EXPECTATIVAS DEL NEGOCIO

En resumen: Los usuarios de QualityOps AI son el desarrollador que propone cambios, el responsable de integrar a la rama principal y el evaluador que necesita evidencia de calidad. El negocio espera detectar problemas **antes** de integrar, con retroalimentación en menos de un minuto, sin costo de licencias y con datos que nadie pueda maquillar. Las expectativas se tradujeron en indicadores medibles; todas se cumplen con los datos actuales salvo la eficacia sobre defectos críticos, que no tiene datos.

7.1 Usuarios y necesidades

Tabla 28. Usuarios de la herramienta

Usuario	Necesidad	Cómo la atiende QualityOps AI
Desarrollador	Saber si su cambio rompe algo antes de pedir revisión	Ejecuta <code>python -m qualityops</code> en local o ve el check del PR
Responsable de integración (líder técnico)	No integrar código que baje la calidad	Gate obligatorio por ruleset; dictamen de liberación
Evaluador / profesor	Verificar que las métricas son reales y reproducibles	Datos del historial de Git, comandos públicos y evidencias
Equipo que adopta la herramienta	Integrarla sin reescribir su proyecto	<code>--repo</code> sobre cualquier repositorio Python con Git

7.2 Historias de usuario

- **HU-01.** Como desarrollador, quiero ver en el Pull Request si mis pruebas, cobertura y complejidad cumplen los umbrales, para corregir antes de pedir revisión. *Criterio:* el resumen del run muestra la tabla del gate (E08).
- **HU-02.** Como responsable de integración, quiero que GitHub impida el merge si el gate falla, para que ningún cambio que baje la calidad llegue a main. *Criterio:* ruleset activo y un PR bloqueado (E08b, demostración de bloqueo).
- **HU-03.** Como evaluador, quiero un informe con formato fijo y un dictamen, para decidir la liberación sin interpretar datos crudos. *Criterio:* informe de 12 secciones en CI (E10k).
- **HU-04.** Como evaluador, quiero que los defectos y tiempos salgan del historial, para confiar en que no fueron inventados. *Criterio:* `python -m qualityops.defects` reproduce los defectos (E11f).

- **HU-05.** Como líder técnico, quiero una explicación en lenguaje natural de las métricas, para comunicar riesgos a quien no es técnico. *Criterio:* interpretación del LLM en el informe (E10i).

7.3 Expectativas del negocio e indicadores

Tabla 29. Expectativas del negocio, indicador, meta y valor real

Expectativa	Indicador	Meta	Valor real	¿Cumple?
Detectar problemas antes de integrar	Defectos atrapados antes de producción / con fase conocida	≥ 85 % (DRE promedio, Jones 2008)	100.0 % (3/3)	Sí
Retroalimentación rápida	Duración del pipeline por PR	< 1 min	34–45 s en las ejecuciones capturadas	Sí
Reparar rápido	MTTR (mediana)	< 1 h (élite DORA)	0.05 h (n = 3)	Sí
Código mantenible	Funciones con CC ≤ 10	100 %	87/87	Sí
Código verificable	Cobertura de líneas	≥ 75 %	88.73 %	Sí
Datos confiables	Verificaciones de coherencia correctas	5/5	5/5	Sí
Costo cero	Costo de licencias y servicios	\$0	GitHub (repositorio público) y Gemini (nivel gratuito)	Sí
Revisión efectiva	Eficacia de la revisión	Informativa	66.67 %	—
Contener errores críticos	Eficacia de pruebas sobre críticos	≥ 85 %	Sin datos (0 críticos)	Sin datos

7.4 Requisitos implícitos

Además de lo explícito, el usuario espera cosas que nadie escribe pero que se notan si faltan:

- **No exponer secretos.** La llave de la API nunca se escribe en archivos ni en el chat; se carga con `Read-Host` en local y como secreto en GitHub.
- **No fallar por causas externas.** Sin red o sin llave, el programa sigue funcionando con la ruta por reglas.
- **Mensajes claros.** Cada falla del gate dice qué criterio falló, su valor y el umbral.
- **No inventar valores.** Con n = 0 el programa escribe «sin datos» o `null`, nunca un 0 % o un 100 % engañoso.

8. ASEGURAMIENTO DE LA CALIDAD DE SOFTWARE

En resumen: Apliqué un plan SQA resumido según IEEE 730-2014: cada cambio pasa por linter, pruebas, métricas, quality gate, revisión en el Pull Request y CI obligatoria antes de integrarse; las configuraciones se controlan con Git; y los defectos se registran en el historial. El resultado: 16 PRs integrados con CI, 56 pruebas, cobertura 88.73 % y 3 defectos de producto detectados antes de producción.

8.1 Plan SQA según IEEE 730

Tabla 30. Actividades del plan SQA

Actividad	Cuándo	Herramienta	Criterio de aceptación	Evidencia
Revisión automática de código	Cada commit (local) y cada PR (CI)	ruff	0 errores	E04, E08d
Pruebas unitarias e integración	Cada commit y cada PR	pytest	0 fallidas, 0 errores	E05, E10h
Medición de cobertura y complejidad	Cada PR	pytest-cov, radon	$\geq 75\%$, $CC \leq 10$	E08
Quality gate	Cada PR	quality_gate.py	Código 0	E08a
Revisión del PR (autorrevisión con checklist)	Antes de cada merge	Plantilla de PR	Checklist completo	docs/pull_request_template.md
Gestión de configuración	Siempre	Git, ramas, ruleset	Sin commits directos a main	E08b, tabla de PRs
Registro y análisis de defectos	En cada <code>fix</code>	Trailers + defects.py	Severidad y fase en cada fix	E15c, E11f
Informe de calidad	Cada PR	report.py	12 secciones + dictamen	E10k
Registro de decisiones y uso de IA	En cada decisión	DECISIONES.md, AI_LOG.md	Motivo y alternativa	docs/

8.2 Gestión de configuración

Ningún cambio llegó a `main` sin Pull Request después de activar el ruleset. La tabla siguiente se obtiene con `git log --merges --first-parent` y muestra cada integración con su hora.

Tabla 31. Pull Requests integrados a main (git log --merges --first-parent)

Merge	Fecha y hora	Pull Request y rama
9d93529	2026-10-08 08:37	PR #43 — rama T20-diagramas
ae04761	2026-10-08 08:04	PR #41 — rama T19b-registro
acd773f	2026-10-08 03:57	PR #40 — rama T17b-gemini
78be41c	2026-10-08 02:38	PR #39 — rama T18-T19-dashboard
054a942	2026-10-08 01:39	PR #38 — rama T12-T17-informe-ia
be6bb9c	2026-10-08 01:17	PR #37 — rama T15-estimacion
03ca184	2026-10-08 00:38	PR #36 — rama T13-T14-proceso-proyecto
00517a7	2026-10-08 00:17	PR #35 — rama T11-ci
ffb14d0	2026-10-06 12:49	PR #34 — rama T10-quality-gate
7f7f38e	2026-10-06 12:47	PR #33 — rama T09-pyproject
454864e	2026-10-06 12:31	PR #32 — rama T08-cli
2e9abd5	2026-10-05 21:35	PR #31 — rama T07-densidad
38ea0c8	2026-10-05 21:22	PR #30 — rama T06-defectos-szz
c7edea1	2026-10-05 10:44	PR #29 — rama T05-cobertura
8fa7528	2026-10-05 10:23	PR #28 — rama T04-complejidad
3a8a2d5	2026-10-05 00:55	PR #26 — rama T02-kanban

8.3 Revisiones técnicas

Cada PR se revisa con la plantilla `docs/pull_request_template.md`: código comentado, ruff limpio, pytest en verde, prueba nueva cuando aplica, sin secretos y registro del uso de IA. Si la revisión encuentra un defecto, se corrige en el mismo PR con un commit `fix:` y el trailer `Detectado-en: revision`.

Con esa regla, la revisión encontró **2 de 3** defectos (66.67 %). El defecto 1 lo detecté al leer el informe en el resumen del PR #38: `__main__.py` se mostraba como «**main.py**» porque Markdown interpreta los guiones bajos como negritas.

```
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> gh pr checks --watch
All checks were successful
0 cancelled, 0 failing, 1 successful, 0 skipped, and 0 pending checks

+-----+-----+-----+-----+
| NAME | DESCRIPTION | ELAPSED | URL |
+-----+-----+-----+-----+
| CI - QualityOps/ca... | 42s | https://github... |
+-----+-----+-----+-----+
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> gh pr merge --merge --delete-branch
non-200 OK status code: 502 Bad Gateway body: "<html>\r\n<head><title>502 Bad Gateway</title></head>\r\n<body>\r\n<center><h1>502 Bad Gateway</h1></center>\r\n<hr><center>nginx</center>\r\n</body>\r\n</html>\r\n"
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> git switch main

Switched to branch 'main'
Your branch is up to date with 'origin/main'.
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> git pull
Already up to date.
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> |
```

Figura 31. Defecto 1 detectado en la revisión del PR #38 (evidencia E15)

Qué muestra. La sección 10 del informe en Actions con el nombre del archivo mal mostrado.

Qué demuestra. La revisión del PR es una fase real de detección de defectos, no un trámite (MET-J1).

8.4 Verificación y validación

La **verificación** comprueba que el programa calcula bien: pruebas automatizadas con valores conocidos y comparación contra herramientas de referencia (radon, coverage). La **validación** comprueba que resuelve el problema: el pipeline bloquea cambios malos y el informe permite decidir la liberación.

Tabla 32. Pruebas automatizadas por archivo

Archivo de pruebas	Pruebas
<code>test_ai_advisor_report.py</code>	13
<code>test_cli.py</code>	1
<code>test_coverage_metrics.py</code>	3
<code>test_dashboard.py</code>	5
<code>test_defects.py</code>	5
<code>test_density.py</code>	3
<code>test_estimation.py</code>	6
<code>test_process_project_metrics.py</code>	7
<code>test_product_metrics.py</code>	5

Archivo de pruebas	Pruebas
test_quality_gate.py	8
Total	56

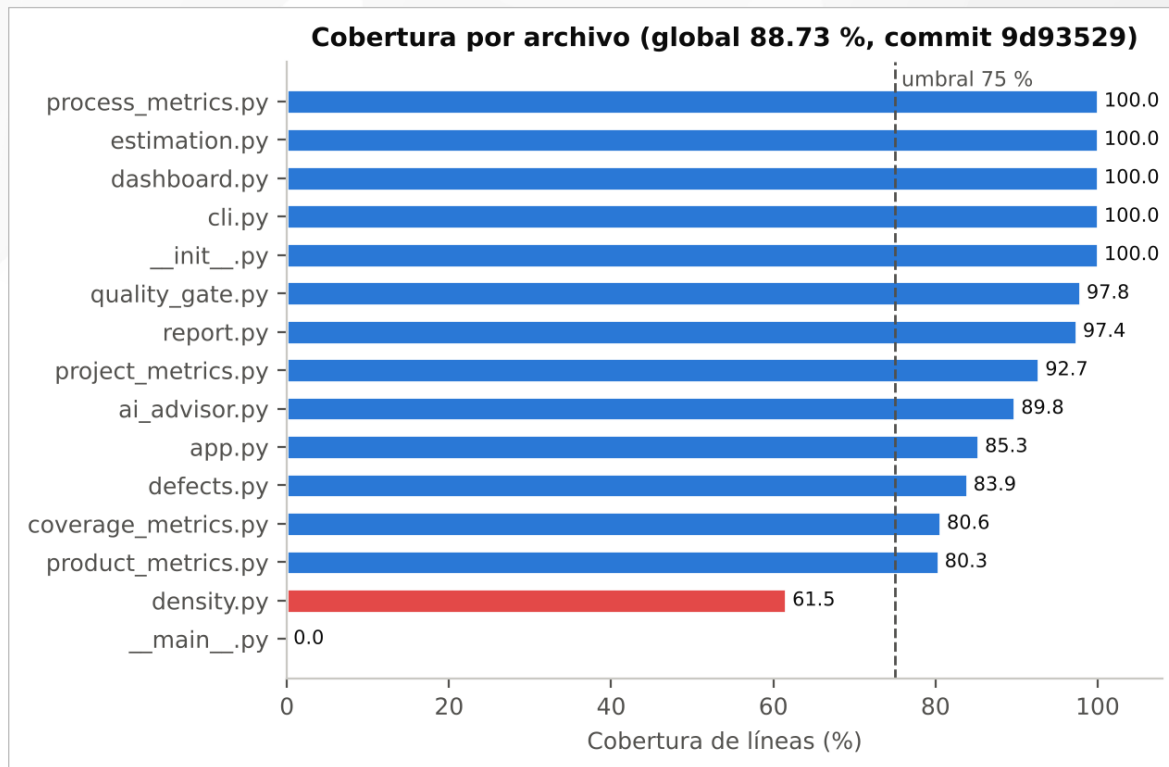


Figura 32. Cobertura de líneas por archivo frente al umbral del gate

Qué muestra. El porcentaje de líneas ejecutadas por las pruebas en cada archivo de producto (commit 9d93529). La línea punteada es el umbral del gate (75 %); en rojo, los archivos por debajo.

Cómo leerla. El umbral se aplica a la cobertura **global**, no por archivo. `density.py` (61.5 %) queda abajo porque su función de línea de comandos no se prueba, y `__main__.py` (0 %) solo contiene la llamada de arranque `main()`; ambos son riesgos bajos, pero quedan señalados para mejorar.

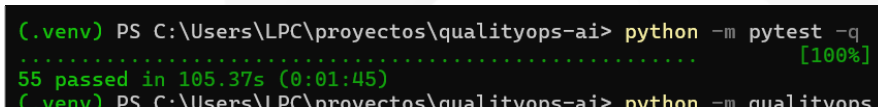
Cómo se generó. `python docs/reporte/graficas.py` (función `cobertura_por_archivo`).

8.5 Defectos registrados

Tabla 33. Defectos de producto minados del historial

Fix	Descripción	Severidad	Fase	Inductor	MTTD h	MTTR h
a6139f9	nombres de archivo con guion bajo se mostraban en negritas en el informe	menor	revision	cf916cb	0.22	0.13
f82e42f	elimina el proveedor GitHub Models, retirado por GitHub el 2026-07-30	mayor	pruebas	c8b2162	0.23	0.05
a02edd4	el informe y el dashboard explican si el LLM se usó, falló o no está configurado	menor	revision	c8b2162	0.25	0.03

Además hubo **1 corrección excluida** de las métricas de producto: [71f84cb](#) (fix(tests): aislar test_main_escribe_archivos de la API real de Gemini) — motivo: no modifica código de producto. Se trata del defecto 4: una prueba llamaba a la API real de Gemini cuando la llave estaba cargada, lo que hacía que pytest tardara 105 s. Es un defecto real del proceso de pruebas, pero no del producto, por eso la densidad no lo cuenta.



```
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> python -m pytest -q
..... [100%]
55 passed in 105.37s (0:01:45)
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> python -m qualityops
```

Figura 33. Defecto 4: pytest tardaba 105 segundos (evidencia E10j)

Qué muestra. [56 passed in 105.37s](#): una prueba esperaba la respuesta de la API real.

Qué demuestra. Medir también la duración de las pruebas revela defectos que el resultado verde no muestra; tras la corrección (commit [71f84cb](#)) las mismas pruebas tardan 30 s (E10l).

9. CARACTERÍSTICAS DE CALIDAD: IDONEIDAD FUNCIONAL Y FIABILIDAD

En resumen: Evalué las dos características que pide la actividad con el modelo ISO/IEC 25010:2023 y medidas al estilo de ISO/IEC 25023. **Idoneidad funcional:** 15 de 15 requisitos funcionales implementados con evidencia, 56 de 56 pruebas correctas y ninguna función sin requisito. **Fiabilidad:** 3 defectos de producto, todos corregidos antes de producción; el pipeline terminó con éxito en 94.7 % (18 de 19 ejecuciones); el programa tolera las fallas del servicio de IA; y la recuperación mediana fue de 0.05 h.

9.1 Modelo de calidad aplicado

ISO/IEC 25010:2023 describe la calidad del producto con nueve características. La actividad pide dos: la **idoneidad funcional** (el programa hace lo que debe y lo hace bien) y la **fiabilidad** (lo sigue haciendo bajo las condiciones esperadas y ante fallas). Para cada subcaracterística definí una medida cuantitativa, su fórmula y la fuente del dato, siguiendo la forma de las medidas de ISO/IEC 25023 (una razón entre lo observado y lo esperado).

9.2 Idoneidad funcional

Tabla 34. Medidas de idoneidad funcional

Subcaracterística	Pregunta	Medida	Valor	Evidencia
Complejidad funcional	¿Están todas las funciones especificadas?	RF implementados con evidencia / RF especificados	15/15 = 100 %	Catálogo RF (cap. 6)
Corrección funcional	¿Los resultados son correctos?	Pruebas aprobadas / pruebas ejecutadas; valores iguales a la herramienta de referencia	56/56 = 100 %; CC 7/7 = radon; cobertura = coverage	E07, E06, E10h
Corrección funcional (coherencia)	¿Las métricas son consistentes entre sí?	Verificaciones correctas / verificaciones	5/5 = 100 %	Sección 11 del informe
Pertinencia funcional	¿Cada función sirve a un objetivo del usuario?	Módulos con requisito asociado / módulos funcionales	14/14 = 100 % (0 funciones sin requisito)	Tabla de trazabilidad (cap. 6)

La **complejidad** es 100 % porque cada requisito funcional del catálogo tiene un módulo y una evidencia (no hay funciones prometidas sin implementar). La **corrección** no depende solo de que las pruebas pasen: los valores se compararon contra radon y coverage, y en cada ejecución se comprueba que la cobertura, el KLOC, la densidad y el conteo de pruebas sean aritméticamente consistentes. La **pertinencia** se cuida con la regla de no agregar funciones que no aporten a un requisito: los 14 módulos funcionales (todos menos `__init__.py`) aparecen en la tabla de requisitos.

9.3 Fiabilidad

Tabla 35. Medidas de fiabilidad

Subcaracterística	Medida	Valor	Evidencia
Ausencia de fallos (madurez)	Defectos corregidos / defectos detectados; densidad de defectos	3/3 = 100 %; 2.81 def/KLOC	E11f
Disponibilidad	Ejecuciones exitosas del pipeline / ejecuciones completadas	94.7 % (18 de 19 ejecuciones)	data/ci_runs.json, E16b
Tolerancia a fallos	Fallas del servicio de IA manejadas sin interrumpir el informe / fallas probadas	4/4 (sin llave, error 503, tiempo agotado, modelo inexistente 404)	E10g, E10m, E10l, pruebas
Recuperabilidad	Tiempo medio de restauración (MTTR): merge – fix	Mediana 0.05 h; promedio 0.07 h (n = 3)	E11f

La **madurez** se apoya en dos datos: los tres defectos de producto se corrigieron antes de integrarse a una versión liberada y la densidad global es baja frente al rango de 1 a 25 defectos por KLOC que McConnell (2004) reporta para software entregado. La **disponibilidad** se mide sobre el pipeline, que es el servicio que la herramienta presta al equipo; una ejecución fallida por el quality gate cuenta como falla aunque sea un bloqueo intencional, por lo que el valor es conservador. La **tolerancia a fallos** se demostró con fallas reales del servicio de Gemini durante el desarrollo, no solo simuladas. La **recuperabilidad** usa el MTTR: cuánto tarda en restaurarse `main` después de corregir un defecto.

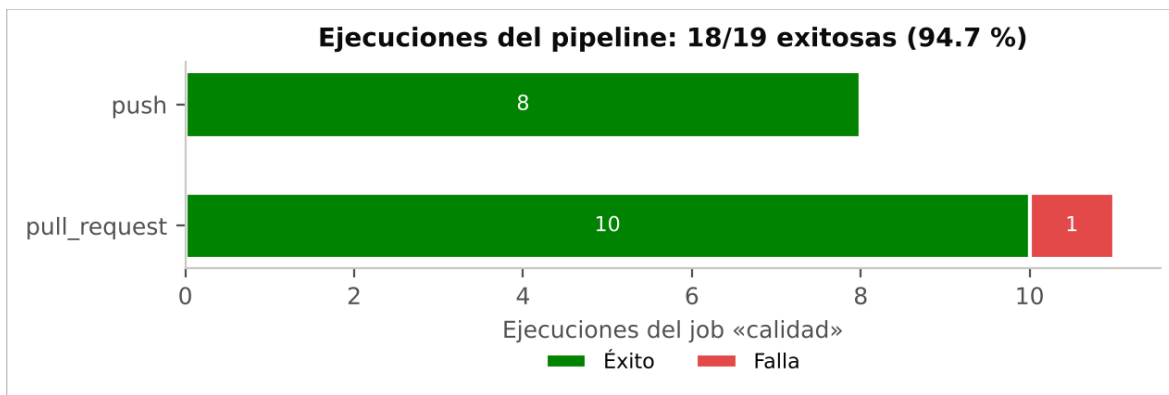


Figura 34. Resultado de las ejecuciones del pipeline por tipo de evento

Qué muestra. Cuántas ejecuciones del job «calidad» terminaron en éxito, falla o cancelación, separadas por evento (pull_request o push).

Cómo se obtuvo. `gh run list -L 100 --json ... > data/ci_runs.json` y `python docs/reporte/graficas.py`.

Qué demuestra. La disponibilidad del servicio de validación con datos de GitHub, no estimados.

```
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> gh run list --json databaseId,event,status,conclusion,headBranch,createdAt,updatedAt > data/ci_runs.json
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> gh run list -L 20
STATUS TITLE WORK... BRANCH EVENT ID ELAPSED AGE
✓ Merge pull... CI -... main push 3779... 47s abou...
✓ T20 - Diag... CI -... T20-... pull... 3779... 59s abou...
X DEMO - blo... CI -... demo... pull... 3779... 55s abou...
✓ Merge pull... CI -... main push 3778... 1m42s abou...
✓ T19b - Reg... CI -... T19b... pull... 3778... 43s abou...
✓ Merge pull... CI -... main push 3778... 56s abou...
✓ T17b - AI... CI -... T17b... pull... 3775... 45s abou...
✓ Merge pull... CI -... main push 3775... 43s abou...
✓ T18/T19 - ... CI -... T18-... pull... 3775... 46s abou...
✓ T18/T19 - ... CI -... T18-... pull... 3774... 46s abou...
✓ Merge pull... CI -... main push 3774... 39s abou...
✓ T12/T17 - ... CI -... T12-... pull... 3774... 44s abou...
✓ T12/T17 - ... CI -... T12-... pull... 3774... 45s abou...
✓ Merge pull... CI -... main push 3774... 39s abou...
✓ T15 - Cuat... CI -... T15-... pull... 3774... 42s abou...
✓ Merge pull... CI -... main push 3773... 42s abou...
✓ T13/T14 - ... CI -... T13-... pull... 3773... 34s abou...
✓ Merge pull... CI -... main push 3773... 40s abou...
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai>
```

Figura 35. Historial de ejecuciones del pipeline (gh run list) (evidencia E16b)

Qué muestra. Las ejecuciones más recientes del workflow con su estado, rama, evento y duración.

Qué demuestra. Es la fuente de la medida de disponibilidad.

```
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> python -m pytest -q
56 passed in 30.36s
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> git add tests/test_ai_advisor_report.py
warning: in the working copy of 'tests/test_ai_advisor_report.py', LF will be replaced by CRLF the next time Git touches it
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> git commit -m "fix(tests): aislar test_main_escribe_archivos de la API real de Gemini" -m "La prueba solo q
uitaba ANTHROPIC_API_KEY; con GEMINI_API_KEY cargada llamaba a Google y tardaba 105 s." -m "Severidad: menor
>> Detectado en: pruebas
>> Evidencia: evidence/E10j_pytest_105s_2026-10-08.png
>> Refs: #17"
[T17b-gemini T1f84cb] fix(tests): aislar test_main_escribe_archivos de la API real de Gemini
1 file changed, 68 insertions(+)
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> $env:QUALITYOPS_MODELO_GEMINI = "gemini-3.8-flash-lite"
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> python -m qualityops.report
Informe generado: reports\quality_report.md · dictamen: LIBERAR CON CONDICIONES · interpretación: reglas
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> Get-Content reports\ai_analysis.json -Encoding UTF8 | Select-String "fuente"|"proveedor"|"error_llm"

"fuente": "reglas",
"error_llm": "RuntimeError: HTTP 404 de Gemini:
{\"error\":{\"message\":\"Model 'gemini-3.8-flash-lite' not found.
Did you mean 'gemini-3.5-flash-lite'? Please verify the model name
against the supported list: https://ai.google.dev/gemini-api/docs/mod
els\",\"code\":\"not_found\"}}"
```

Figura 36. Corrección del defecto 4 y modelo inexistente reportado con su detalle (evidencia E10l)

Qué muestra. pytest con 56 pruebas en 30 s después de aislar la prueba, el commit `fix(tests)` con sus trailers y un intento con el modelo `gemini-3.8-flash-lite`, que no existe: el programa reporta el HTTP 404 con el mensaje de Google y la sugerencia de modelo, y usa la ruta por reglas.

Qué demuestra. Ante un error de configuración el programa no se detiene y explica la causa (tolerancia a fallos).

10. ENFOQUE PREVENTIVO

En resumen: El enfoque preventivo consiste en evitar que un defecto llegue a la rama principal en lugar de corregirlo después. En QualityOps AI la prevención es automática: el quality gate se ejecuta en cada Pull Request y el ruleset impide integrar si falla. Lo demostré bloqueando un cambio real con complejidad 11. Los 3 defectos de producto se detectaron en revisión o en pruebas, ninguno en producción.

10.1 Concepto

ISO/IEC/IEEE 90003:2018 (aplicación de ISO 9001 al software) orienta la gestión de la calidad a la prevención: controlar el proceso para que el producto salga bien, en lugar de inspeccionar al final. En DevOps esta idea se llama *shift-left*: mover la verificación lo más a la izquierda (lo más temprano) posible del ciclo de vida. La razón es económica: corregir un defecto cuesta más cuanto más tarde se detecta (Boehm, 1981).

10.2 Mecanismos preventivos implementados

Tabla 36. Controles preventivos y el problema que evitan

Control	Cuándo actúa	Qué evita	Evidencia
Versiones fijas de dependencias	Instalación	Que un cambio externo rompa el build sin aviso	requirements.txt
ruff (linter)	Antes de las pruebas	Errores de sintaxis, imports sin usar, malas prácticas	E08d
Pruebas automatizadas	Cada PR	Regresiones en funciones ya probadas	E10h
Umbral de complejidad ($CC \leq 10$)	Cada PR	Funciones difíciles de probar y mantener	E08a, demostración de bloqueo
Umbral de cobertura ($\geq 75\%$)	Cada PR	Código nuevo sin pruebas	E08a
Ruleset proteger-main	Al intentar el merge	Integrar con el check en rojo o sin PR	E08b
Revisión con checklist	Antes del merge	Defectos de presentación o lógica que las pruebas no ven	E15
Verificaciones de coherencia	Cada informe	Publicar métricas contradictorias	Sección 11 del informe
Respaldo por reglas del AI Advisor	Cada informe	Que una caída de la IA deje sin informe al equipo	E10m
Secretos fuera del código	Siempre	Fuga de credenciales	ci.yml

10.3 Demostración de bloqueo

Para probar que el gate realmente impide integrar, hice dos demostraciones. La primera fue local, con un umbral de cobertura de demostración (95 %) que la herramienta no alcanzaba (capítulo 6, quality gate). La segunda fue en GitHub: abrí un Pull Request que agrega [qualityops/demo_bloqueo.py](#), una función con una

cadena de 10 `if/elif` (complejidad ciclomática 11). El archivo pasa el linter; solo el quality gate lo detiene. El PR se cerró sin merge.

```
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> gh pr create --title "T19b - Registro de tiempos, evidencias y matriz" --body "Registro posterior a PR #39 y #40. Matriz 21/44 (47.7 %)."
```

Creating pull request for T19b-registro into main in TaikoVC/qualityops-ai

<https://github.com/TaikoVC/qualityops-ai/pull/41>

```
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> gh pr checks --watch
```

no checks reported on the 'T19b-registro' branch

```
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> gh pr merge --merge --delete-branch
```

✗ Pull request TaikoVC/qualityops-ai#41 is not mergeable: the base branch policy prohibits the merge. To have the pull request merged after all the requirements have been met, add the '--auto' flag. To use administrator privileges to immediately merge the pull request, add the '--admin' flag.

```
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> git switch main
```

Switched to branch 'main'

Your branch is up to date with 'origin/main'.

```
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> git pull
```

Already up to date.

```
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> |
```

Figura 37. El ruleset impide el merge mientras el check obligatorio no termina (PR #41) (evidencia E08g)

Qué muestra. Al intentar el merge del PR #41 un segundo después de crearlo, GitHub respondió «the base branch policy prohibits the merge»: el check «calidad» todavía no terminaba.

Qué demuestra. Ni el dueño del repositorio puede integrar sin el check en verde (salvo con `--admin`, que no se usó). El PR se integró después, cuando el pipeline terminó en verde.

```
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> gh pr create --title "DEMO - bloqueo del quality gate (no integrar)" --body "Demostración de CAL-07: CC 11 > umbral 10. Se cierra sin merge."
```

Warning: 2 uncommitted changes

Creating pull request for demo-bloqueo-gate into main in TaikoVC/qualityops-ai

<https://github.com/TaikoVC/qualityops-ai/pull/42>

```
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> gh pr checks --watch
```

Some checks were not successful

0 cancelled, 1 failing, 0 successful, 0 skipped, and 0 pending checks

NAME	DESCRIPTION	ELAPSED	URL
✗ CI - QualityOps/calid...		49s	https://github.com/TaikoVC/qualityops-ai/pull/42

```
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> |
```

Figura 38. Check «calidad» en rojo para el PR de demostración (evidencia E17a)

Qué muestra. `gh pr checks --watch` del PR de demostración con el check fallido.

Qué demuestra. El pipeline detecta el problema sin intervención humana.

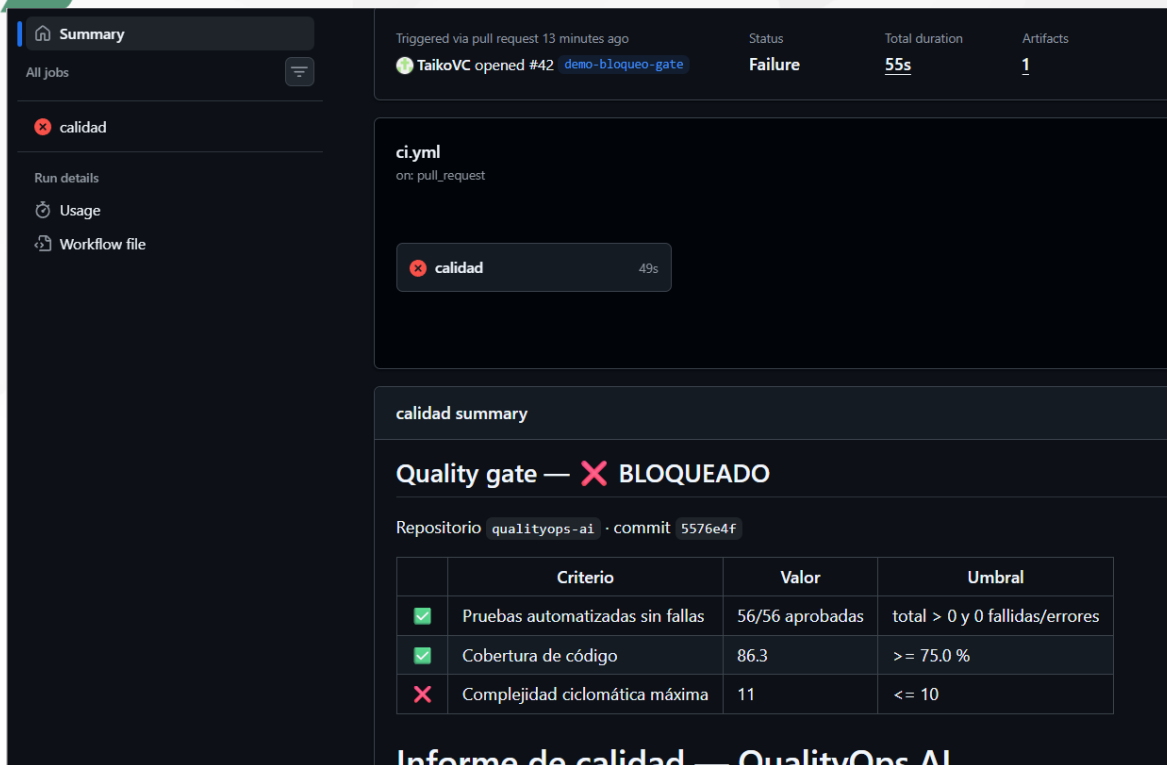


Figura 39. Resumen del run bloqueado: complejidad 11 > 10 (evidencia E17b)

Qué muestra. El resumen del job con «Quality gate — BLOQUEADO» y el criterio de complejidad máxima en falla (valor 11, umbral ≤ 10). El informe de calidad se generó de todos modos con dictamen NO LIBERAR.

Qué demuestra. El bloqueo es explicado: el desarrollador sabe exactamente qué corregir.

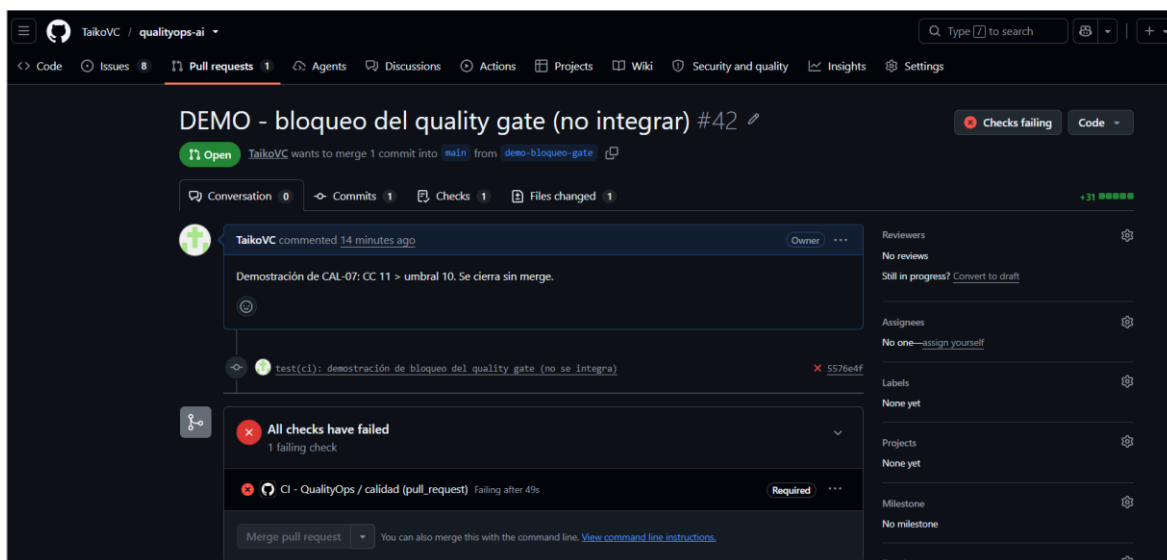


Figura 40. GitHub impide el merge del PR bloqueado (evidencia E17c)

Qué muestra. La sección de merge del PR con el check requerido «calidad» fallido y el botón de merge deshabilitado.

Qué demuestra. La prevención no depende de la disciplina del autor: la plataforma la impone (CAL-07).

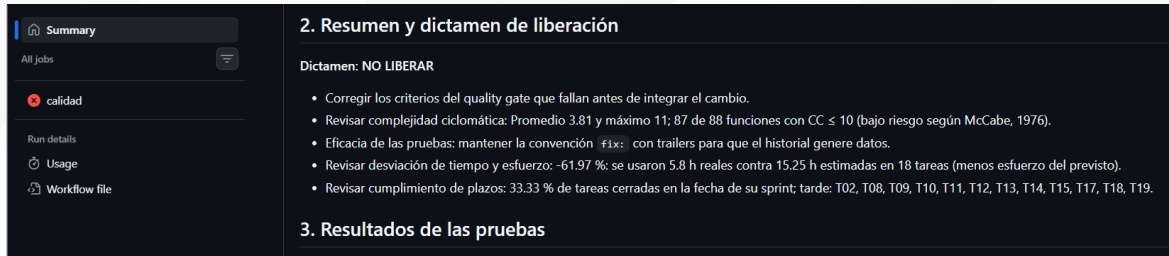


Figura 41. Informe del PR bloqueado: dictamen NO LIBERAR (evidencia E17d)

Qué muestra. La sección 2 del informe del PR #42 con el dictamen NO LIBERAR y la recomendación «Corregir los criterios del quality gate que fallan antes de integrar el cambio», además de la valoración de la complejidad (máximo 11; 87 de 88 funciones con $CC \leq 10$).

Qué demuestra. El informe se genera aunque el gate bloquee (`if: always()`) y explica la causa.

10.4 Detección temprana de los defectos reales

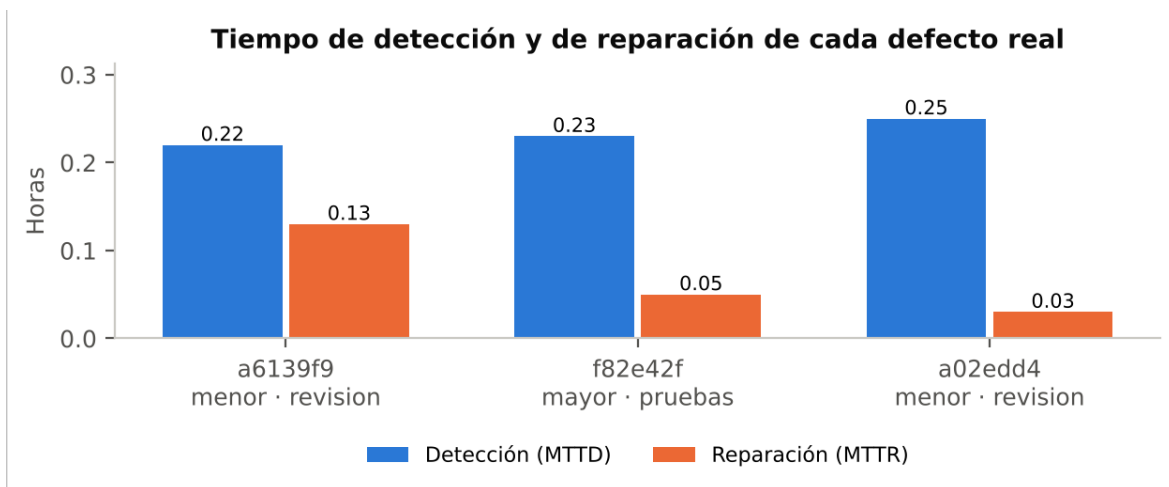


Figura 42. Tiempo de detección y de reparación de cada defecto real

Qué muestra. Para cada defecto de producto, el tiempo desde el commit que lo introdujo hasta su corrección (azul) y desde la corrección hasta el merge (naranja), con su severidad y la fase en que se detectó.

Cómo leerla. Ningún defecto vivió más de 15 minutos antes de corregirse, y todos se detectaron en revisión o en pruebas, antes del merge. El defecto 2 (mayor) se detectó porque el pipeline ejecutó el AI Advisor contra el servicio real; en un proceso sin CI habría aparecido al entregar.

Cómo se generó. `python docs/reporte/graficas.py` (función `tiempos_defectos`) sobre `metrics.json`.

La conclusión del capítulo es que el costo de los defectos fue mínimo porque se detectaron en la misma sesión en que se introdujeron. Esa es la meta del enfoque preventivo: acortar la distancia entre el error y su detección.

11. CARÁCTER SISTEMÁTICO

En resumen: El proceso es sistemático porque cada cambio sigue los mismos pasos (issue → rama → commits convencionales → PR → CI → merge → registro), porque cada requisito se rastrea hasta su evidencia, porque las normas aplicadas están documentadas con su aplicación concreta y porque cada decisión importante quedó registrada con su motivo y la alternativa descartada.

11.1 Ciclo de trabajo repetible

1. Mover la tarjeta de la tarea a *In Progress* y anotar la hora de inicio.
2. Crear la rama `TXX-descripcion` desde `main` actualizado.
3. Escribir código y pruebas; ejecutar `ruff check .` y `python -m pytest` en local.
4. Hacer commits con Conventional Commits (`feat:`, `fix:` con trailers, `docs:...`).
5. Abrir el Pull Request con la plantilla y esperar el check «calidad».
6. Revisar el resumen del run; si hay defecto, corregirlo en la misma rama con `fix:` y `Detectado-en: revision`.
7. Hacer merge con merge commit, borrar la rama y anotar la hora de fin en `time_log.csv`.
8. Registrar evidencia (`EVIDENCIAS.md`), actualizar la matriz y la bitácora de IA.

11.2 Trazabilidad de extremo a extremo

Tabla 37. Ejemplos reales de trazabilidad requisito → evidencia

Requisito	Tarea / issue	PR / merge	Código	Pruebas	Evidencia
CAL-07 Enfoque preventivo	T10 / #10	#34 / ff14d0	quality_gate.py	test_quality_gate.py (8)	E08a, E08b
MET-J1 Eficacia de revisión	T14 / #14	#36 / 03ca184	project_metrics.py	test_process_project_metrics.py (7)	E11d, E11f
PRG-02 IA en frameworks	T17 / #17	#38 / 054a942 y #40 / acd773f	ai_advisor.py	test_ai_advisor_report.py (13)	E10h, E10k
MET-P1 Complejidad	T04 / #4	#28 / 8fa7528	product_metrics.py	test_product_metrics.py (5)	E07

11.3 Normas aplicadas

Una norma solo se cita como aplicada si tiene una aplicación concreta y una evidencia. La tabla proviene de `docs/NORMAS.md`.

Tabla 38. Normas aplicadas, cómo y dónde

Norma	Qué establece	Cómo se aplica	Evidencia
ISO/IEC 25010:2023 — Modelo de calidad del producto (SQuaRE)	9 características de calidad del producto con sus subcaracterísticas	Se evalúan idoneidad funcional (completitud, corrección, pertinencia) y fiabilidad (ausencia de fallos, disponibilidad, tolerancia a fallos, recuperabilidad), cada una con una medida real del proyecto	Tabla ISO 25010 del reporte (T16)

Norma	Qué establece	Cómo se aplica	Evidencia
ISO/IEC 25023:2016 — Medición de la calidad del producto	Medidas cuantitativas para las características de 25010	Base de las fórmulas de la tabla anterior (p. ej., completitud funcional = funciones implementadas / especificadas)	docs/ + reporte
IEEE 730-2014 — Procesos de aseguramiento de la calidad del software	Contenido de un plan SQA: actividades, responsables, estándares, métricas, herramientas	Plan SQA resumido: pruebas, lint, revisión por PR, quality gates, métricas y registros	Capítulo SQA del reporte
ISO/IEC/IEEE 29119-3:2021 — Documentación de pruebas	Estructura de documentos de prueba, incluido el informe de finalización de pruebas	Formato fijo del informe de calidad que genera el programa en cada ejecución	reports/quality_report.md (T12)
ISO/IEC/IEEE 12207:2017 — Procesos del ciclo de vida del software	Procesos técnicos y de gestión del ciclo de vida	Maapeo de cada etapa del pipeline (desarrollo, verificación, integración, gestión de configuración, liberación) a sus procesos	Diagrama del pipeline (T20)
ISO/IEC/IEEE 90003:2018 — Aplicación de ISO 9001:2015 al software	Gestión de calidad basada en procesos, prevención y mejora continua	Fundamento del enfoque preventivo (quality gates antes de integrar) y de la mejora continua (métricas → acciones)	Capítulo de enfoque preventivo
ISO/IEC 20926:2009 — Método IFPUG de medición del tamaño funcional	Conteo de puntos de función (EI, EO, EQ, ILF, EIF)	Técnica de estimación por puntos de función	Tabla de estimación (T15)

11.4 Mapeo con los procesos de ISO/IEC/IEEE 12207

Tabla 39. Procesos del ciclo de vida y su práctica en el proyecto

Proceso (ISO/IEC/IEEE 12207:2017)	Práctica en QualityOps AI
Planificación del proyecto	Backlog T01–T25 con estimación de tres puntos antes del código; sprints con fecha
Evaluación y control del proyecto	Desviación por módulo y cumplimiento de plazos calculados en cada ejecución
Gestión de decisiones	docs/DECISIONES.md (D01–D19) con motivo y alternativa
Gestión de la configuración	Git, una rama por tarea, merge commits, ruleset, versiones fijas
Gestión de la información	metrics.json, informe, evidencias con nombre y fecha
Medición	Motor de métricas de producto, proceso y proyecto
Aseguramiento de la calidad	Plan SQA (cap. 8), matriz de cumplimiento
Implementación e integración	Commits convencionales y PR con CI obligatoria
Verificación	ruff, pytest, comparación con radon/coverage, verificaciones de coherencia
Validación	Quality gate y dictamen de liberación sobre el producto completo
Transición / liberación	Etiqueta vx.Y.Z como versión en producción

11.5 Registro de decisiones

Tabla 40. Decisiones del proyecto (docs/DECISIONES.md, resumidas)

ID	Fecha	Decisión
D01	10-04	QualityOps AI se analiza a sí mismo (dogfooding)
D02	10-04	Métricas calculadas de forma determinista; la IA interpreta
D03	10-04	AI Advisor desacoplado con respaldo por reglas desde el inicio
D04	10-04	GitHub Projects como tablero Kanban
D05	10-04	Estimaciones registradas antes de escribir código
D06	10-04	Motor + CI antes que dashboard
D07	10-04	Defectos obtenidos del historial de Git (minería SZZ reutilizada de la herramienta <code>metrics</code>) + trailers <code>Severidad y Detectado-en</code> en cada <code>fix</code> :
D08	10-04	Reutilizar de la herramienta <code>metrics</code> solo: minería SZZ, utilidades de tabla/guardado y el esquema de resultado por métrica. Reescribir interpretaciones como...
D09	10-04	Informe de calidad con formato fijo basado en ISO/IEC/IEEE 29119-3 (informe de finalización de pruebas) y dictamen de liberación
D10	10-04	Mover el repositorio fuera de <code>C:\Windows\System32</code>
D11	10-05	T01 y T03 se registran como bloque conjunto (21:39–23:02, 1.38 h) repartido en proporción a su estimación (0.5 h c/u → 0.69 h c/u)
D12	10-05	Plantilla de PR en <code>docs/</code> y workflow de CI creado manualmente por el autor
D13	10-05	Posicionar QualityOps AI como herramienta de validación preventiva integrable a cualquier repositorio Python con Git y pytest (las métricas de proceso sirven para...
D14	10-08	El mínimo de 10 hojas aplica a la sección de Requisitos explícitos; el reporte completo además cubre todos los demás puntos

ID	Fecha	Decisión
D15	10-08	Por la fecha límite (jueves 8, 23:00), tareas pequeñas y acopladas pueden cerrarse en un solo PR; su tiempo se reparte en proporción a la estimación (como D11). Los...
D16	10-08	Estimación análoga y por PF con la herramienta <code>metricas</code> como referencia (4.5 h, 103 PF; dato del autor). EST-4 usa la tasa de la referencia (≈ 0.0437 h/PF) sin...
D17	10-08	Ruta LLM del AI Advisor con GitHub Models usando el <code>GITHUB_TOKEN</code> del pipeline (permiso <code>models: read</code> , modelo <code>openai/gpt-4o</code>); la API de Anthropic queda como...
D18	10-08	Se elimina el proveedor GitHub Models (D17 parcialmente revertida): la ejecución real en el CI devolvió una respuesta vacía (<code>JSONDecodeError</code>) y la documentación...
D19	10-08	Proveedor LLM del AI Advisor: Gemini (llave gratuita de Google AI Studio como secreto <code>GEMINI_API_KEY</code> del repositorio), vía API Interactions con la biblioteca...
D20	10-08	Excluir la carpeta <code>docs/</code> del análisis de producto (complejidad, líneas, cobertura y densidad)

12. MÉTRICAS CALCULADAS APLICANDO IA

En resumen: Las nueve métricas se calculan con código determinista y la IA las verifica e interpreta. Producto: complejidad promedio 3.72 (máx. 9), cobertura 88.73 %, densidad 2.81 def/KLOC. Proceso: MTDD 0.23 h, MTTR 0.05 h, eficacia de pruebas 100.0 % (todas las severidades; sin críticos). Proyecto: eficacia de revisión 66.67 %, desviación -61.97 %, plazos 33.33 % a tiempo.

12.1 Cómo participa la IA en las métricas

La IA interviene en dos momentos, con papeles distintos:

Tabla 41. Reparto de responsabilidades entre el código y la IA

Tarea	Quién la hace	Por qué
Calcular cada métrica	Código determinista (radon, pytest-cov, git, fórmulas)	Reproducible: mismo commit, mismo número
Comprobar la consistencia de las métricas	AI Advisor, verificaciones de coherencia	Detecta errores de cálculo antes de publicarlos
Valorar cada métrica contra su referencia	AI Advisor, ruta por reglas	Siempre disponible, sin costo
Redactar interpretación, riesgos y acciones	LLM (gemini) con prompt fijo	Comunica a personas no técnicas
Generar y depurar el código del programa	Claude (asistente de desarrollo)	Acelera el desarrollo; el autor verifica

El prompt que recibe el modelo es fijo y le indica: «Con las métricas siguientes (ya calculadas; NO las recalculas ni inventes otras) escribe en español: 1) una interpretación breve de cada métrica citando su referencia, 2) los 3 riesgos principales y 3) 3 acciones concretas. Si un valor es null o n = 0, dilo explícitamente.»

Después se agregan las verificaciones de coherencia y las métricas en JSON.

Tabla 42. Verificaciones de coherencia de la ejecución analizada

Verificación	Resultado
Cobertura = ejecutadas / ejecutables × 100	Correcta
KLOC = SLOC / 1000	Correcta
Densidad = defectos / KLOC	Correcta
Pruebas: aprobadas + fallidas + errores + omitidas = total	Correcta
Defectos de la minería = defectos usados en la densidad	Correcta

Extracto real de la respuesta de Gemini ([reports/ai_analysis.json](#)), sección de riesgos:

- **Deficiente rendimiento en el cumplimiento de plazos:** El porcentaje de entregas a tiempo es sumamente bajo (33.33%), con 12 de 18 tareas reportadas como tarde, lo que evidencia fallas en la planificación temporal o ejecución a pesar de la sobreestimación general de horas.

- **Desviación severa en la estimación de esfuerzo:** Existe una marcada imprecisión en la estimación inicial frente al esfuerzo real registrado (desviación global del proyecto de -61.97%), con variaciones extremas por módulo (desde +39% en gestión hasta -96% en métricas), lo que desestabiliza la predictibilidad del proyecto.
- **Incertidumbre en la métrica de defectos críticos:** El porcentaje de eficacia de pruebas para defectos críticos es **null** (debido a $n = 0$ antes de producción y con fase conocida), lo que impide validar objetivamente la robustez del software ante fallas de alta severidad.

La IA también se equivoca, y por eso no calcula. En una ejecución anterior (commit 71f84cb, registrada en [docs/AI_LOG.md](#)), el modelo escribió que para los defectos críticos hubo «0 antes de producción y 3 con fase conocida», cuando el dato real era 0 y 0: confundió el total de defectos con los críticos. En la ejecución final lo reportó bien, lo que muestra además que la respuesta del modelo no es determinista. Las cifras del informe no cambian en ningún caso porque vienen del código; el ejemplo justifica la decisión D02 y la regla de leer críticamente cualquier texto generado.

12.2 Métricas de producto

Complejidad ciclomática

Fórmula: $CC = \text{número de decisiones} + 1$, por función. **Referencia:** McCabe (1976): 1–10 bajo riesgo, 11–20 moderado, 21–50 alto, más de 50 no probable. **Resultado:** 87 funciones, promedio 3.72, mediana 3, máximo 9; 87 de 87 en bajo riesgo.

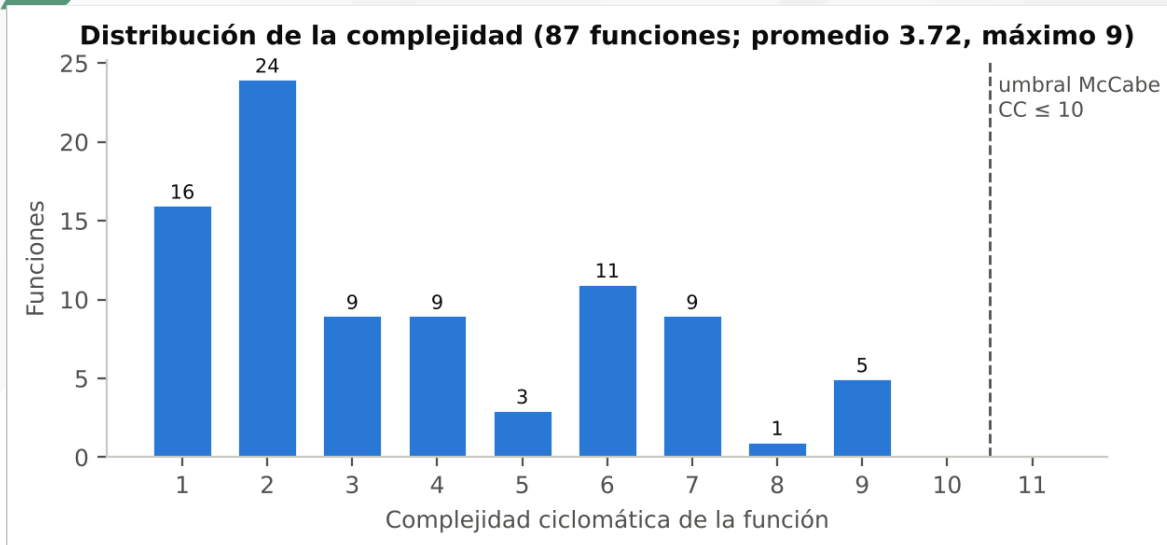


Figura 43. Distribución de la complejidad ciclomática por función

Qué muestra. Cuántas funciones tienen cada valor de complejidad. La línea punteada separa el umbral del gate.

Cómo leerla. La mayoría de las funciones tiene complejidad 1 o 2 (funciones pequeñas con una sola responsabilidad); ninguna cruza el umbral. Las de 8 y 9 son las candidatas a dividir si crecen.

Cómo se generó. `graficas.py` (función `distribucion_complejidad`) con la lista de funciones de `metrics.json`.

Tabla 43. Funciones más complejas

Archivo	Función	Línea	CC	Rango
qualityops/ai_advisor.py	_interp_producto	83	9	B
qualityops/defects.py	_registro_defecto	181	9	B
qualityops/defects.py	minar_defectos	203	9	B
qualityops/estimation.py	estimar	72	9	B
qualityops/report.py	generar_informe	96	9	B
qualityops/dashboard.py	resumen_matriz	35	8	B
qualityops/ai_advisor.py	_recomendaciones	141	7	B
qualityops/ai_advisor.py	texto_de_respuesta_gemini	174	7	B

Cobertura de código

Fórmula: líneas ejecutadas / líneas ejecutables × 100. **Referencia:** Arguelles et al. (2020, Google Testing Blog): 60 % aceptable, 75 % encomiable, 90 % ejemplar.

Resultado: 693 de 781 líneas = **88.73 %** (encomiable). El desglose por archivo está en la gráfica de cobertura del capítulo 8.

Densidad de defectos

Fórmula: defectos de producto / KLOC. **Referencia:** McConnell (2004), 1 a 25 defectos por KLOC en software entregado. **Resultado:** $3 / 1.066 = 2.81 \text{ def/KLOC}$.

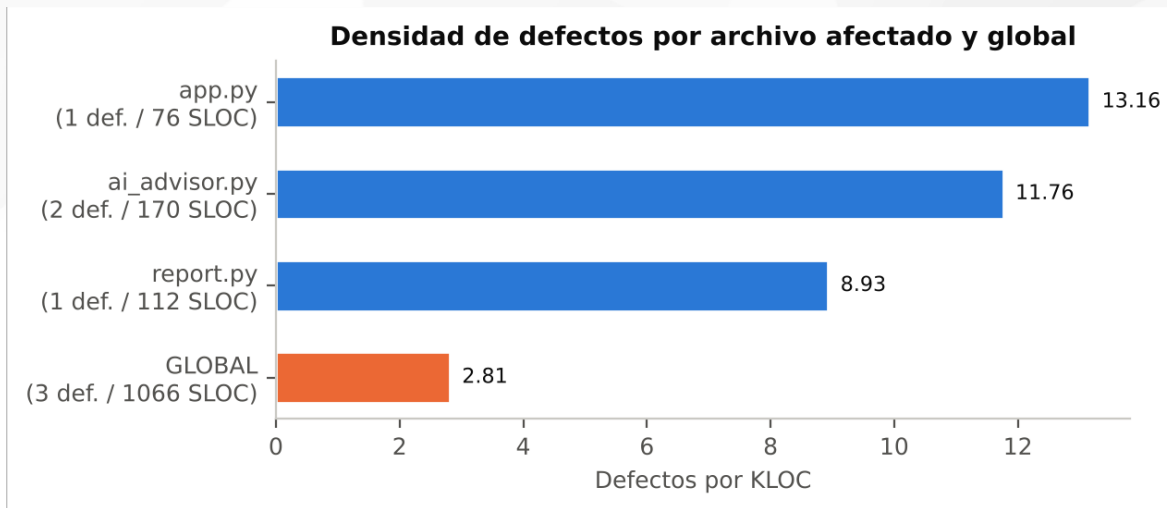


Figura 44. Densidad de defectos por archivo afectado y global

Qué muestra. La densidad de los tres archivos que tuvieron defectos y la densidad global (naranja).

Cómo leerla. La densidad por archivo es alta porque un defecto en un archivo pequeño pesa mucho (1 defecto en 76 SLOC de `app.py` = 13.16). Los defectos se concentran en la capa de presentación e IA (`ai_advisor.py`, `report.py`, `app.py`), la que cambió más el último día; el motor de métricas no tuvo defectos.

Cómo se generó. `graficas.py` (función `densidad_por_archivo`).

12.3 Métricas de proceso

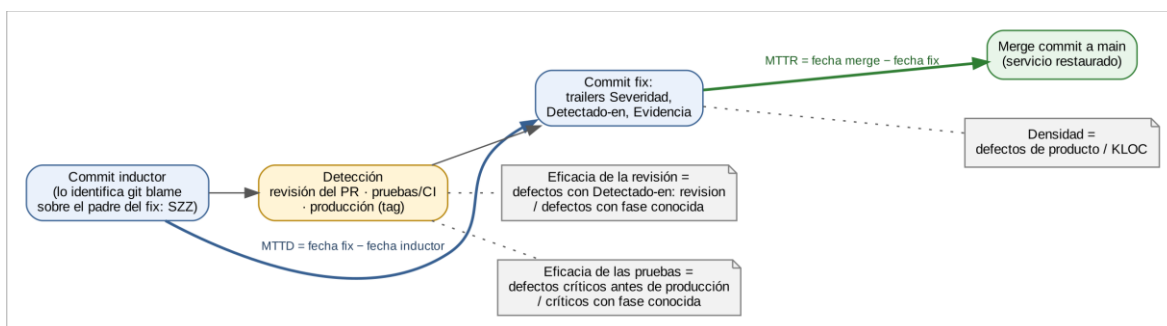


Figura 45. Ciclo de vida de un defecto y dónde se mide cada métrica de proceso

Cómo leerla. Un defecto nace en un commit (inductor), se detecta en una fase (revisión, pruebas o producción), se corrige con un commit `fix`: y se integra con un

merge. El MTDD mide de la introducción a la corrección y el MTTR de la corrección al merge; las eficacias cuentan en qué fase se detectó cada defecto.

Nota. En este proyecto el MTDD se mide hasta la corrección porque la detección y la corrección ocurren en la misma sesión; es una aproximación conservadora (incluye el tiempo de corregir).

Tabla 44. Métricas de proceso

Métrica	Fórmula	Resultado	Referencia
MTDD	promedio y mediana de (fecha fix – fecha inductor)	Promedio 0.23 h, mediana 0.23 h (n = 3)	Informativa
MTTR	promedio y mediana de (fecha merge – fecha fix)	Promedio 0.07 h, mediana 0.05 h (n = 3)	DORA: élite < 1 h
Eficacia de las pruebas (críticos)	críticos antes de producción / críticos con fase conocida	Sin datos (0/0)	Jones (2008): ≈ 85 %
Eficacia de las pruebas (todas)	defectos antes de producción / con fase conocida	100.0 % (3/3)	Jones (2008)

La eficacia sobre errores **críticos** es exactamente lo que pide la actividad, y el programa la calcula, pero el historial no tiene ningún defecto crítico. Reportar 100 % sería engañoso (no se atrapó ninguno porque no hubo); reportar 0 % sería falso. Por eso el valor es «sin datos» y se muestra también la eficacia de todas las severidades.

12.4 Métricas de proyecto

Eficacia de la revisión

Fórmula: defectos detectados en revisión / defectos con fase conocida × 100.

Resultado: 2 de 3 = **66.67 %**. Dos de los tres defectos los encontré revisando el PR antes de integrarlo, y uno lo encontraron las pruebas del pipeline.

Desviación de tiempo y esfuerzo por módulo

Fórmula: (horas reales – horas estimadas) / horas estimadas × 100, por módulo y total, sobre las tareas terminadas. **Resultado:** 18 tareas, 15.25 h estimadas contra 5.8 h reales = **-61.97 %**.

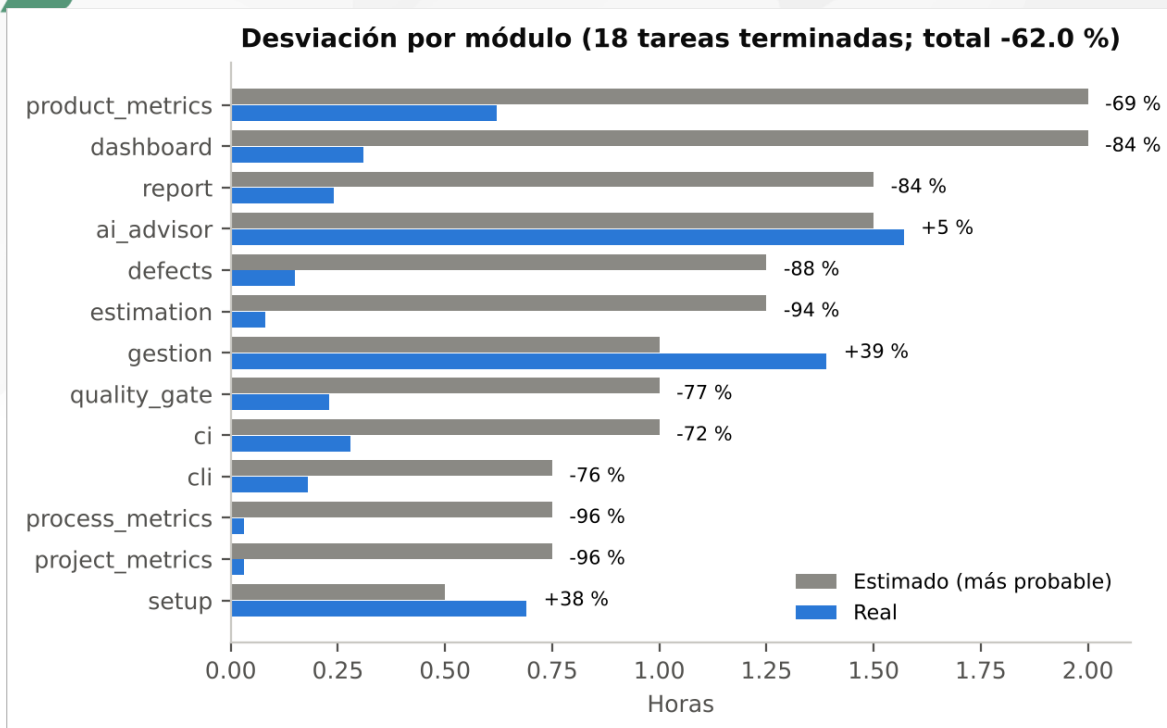


Figura 46. Horas estimadas y reales por módulo

Qué muestra. Para cada módulo, las horas estimadas (valor más probable, gris) y las reales (azul), con el porcentaje de desviación.

Cómo leerla. Tardaron más de lo estimado: `setup` (+38 %), `gestion` (+39 %), `ai_advisor` (+5 %). `setup` y `gestion` fueron tareas manuales (crear el repositorio, moverlo fuera de System32, configurar el tablero); `ai_advisor` incluye el retrabajo de T17b (cambio de proveedor de IA). Los demás módulos tardaron entre 69 % y 96 % menos, porque el código lo generó la IA y el tiempo registrado es el de integrar y verificar (ver amenazas a la validez).

Relación con la densidad de defectos. La actividad pide relacionar la desviación por módulo con la densidad: los módulos con mayor ahorro de tiempo (`process_metrics`, `project_metrics`, `estimation`) no tuvieron defectos, y los defectos aparecieron en la capa de IA y presentación, la de mayor cambio de alcance (D17–D19).

Cómo se generó. `graficas.py` (función `desviacion_por_modulo`).

Cumplimiento de plazos

Fórmula: tareas cerradas en o antes de la fecha de su sprint / tareas evaluadas.

Resultado: 6 de 18 = **33.33 %**. Tareas tarde: T02, T08, T09, T10, T11, T12, T13,

T14, T15, T17, T18, T19. Las fechas planeadas asumían una dedicación diaria que no pude sostener el 6 y 7 de octubre; el trabajo se recuperó el 8, por eso el esfuerzo fue menor al estimado y aun así los plazos se incumplieron: son dos métricas distintas.

12.5 Interpretación del AI Advisor

Tabla 45. Valoración por reglas de cada métrica (ai_analysis.json)

Métrica	Valoración	Interpretación
Complejidad ciclomática	buena	Promedio 3.72 y máximo 9; 87 de 87 funciones con CC $\leq$ 10 (bajo riesgo según McCabe, 1976).
Cobertura de código	encomiable	88.73 % global (Google Testing Blog: 60 aceptable, 75 encomiable, 90 ejemplar). Archivos con menos de 60 %: qualityops/__main__.py .
Densidad de defectos	baja	2.81 defectos/KLOC con 3 defectos registrados.
MTTD	informativa	Mediana 0.23 h (n = 3) entre la introducción y la corrección.
MTTR	élite (DORA)	Mediana 0.05 h (n = 3) del fix al merge.
Eficacia de las pruebas	sin datos	None % de defectos críticos atrapados antes de producción (Jones, 2008: DRE promedio $\approx$ 85 %).
Desviación de tiempo y esfuerzo	alta	-61.97 %: se usaron 5.8 h reales contra 15.25 h estimadas en 18 tareas (menos esfuerzo del previsto).
Cumplimiento de plazos	mejorable	33.33 % de tareas cerradas en la fecha de su sprint; tarde: T02, T08, T09, T10, T11, T12, T13, T14, T15, T17, T18, T19.

13. TÉCNICAS DE ESTIMACIÓN

En resumen: Estimé el proyecto con las cuatro técnicas antes de escribir código. Juicio de expertos: 22.5 h. Tres puntos (PERT): 24.79 ± 1.35 h. Puntos de función: $97 \text{ PF} \times 0.0437 \text{ h/PF} = 4.24$ h. Análoga: 4.79 h. Las dos últimas, basadas en mi productividad real con IA, fueron las más cercanas a las horas registradas.

13.1 Línea base

La estimación quedó registrada en `data/time_log.csv` en el commit 76f14fb (4 de octubre, 23:02), antes de la primera línea de código del producto. Ese orden es lo que da validez a la desviación (D05). La tarea T26 se agregó después y se excluye de la línea base.

13.2 Juicio de expertos

Para cada tarea estimé tres valores con base en mi experiencia: optimista (O), más probable (M) y pesimista (P). El juicio de expertos es la suma de los valores más probables: **22.5 h** para 25 tareas.

Tabla 46. Línea base de estimación por tarea (horas)

Tarea	Descripción	O	M	P	PERT	Real
T01	Repositorio, estructura, entorno y README	0.25	0.5	1.0	0.54	0.69
T02	Tablero Kanban (GitHub Projects) e issues	0.25	0.5	1.0	0.54	0.70
T03	Estimación inicial y plantillas de datos	0.25	0.5	1.0	0.54	0.69
T04	Complejidad ciclomática y LOC (radon)	0.5	1.0	2.0	1.08	0.27
T05	Cobertura de código (pytest-cov)	0.5	0.75	1.5	0.83	0.25
T06	Minería de defectos SZZ adaptada (git + trailers)	0.75	1.25	2.5	1.38	0.15
T07	Densidad de defectos	0.25	0.25	0.5	0.29	0.10
T08	CLI python -m qualityops y metrics.json	0.5	0.75	1.5	0.83	0.18
T09	Configuración ruff/pytest/coverage (pyproject)	0.25	0.25	0.5	0.29	0.06
T10	Quality gate con umbrales	0.5	0.75	1.5	0.83	0.17
T11	Pipeline GitHub Actions	0.5	1.0	2.0	1.08	0.28
T12	Informe de calidad (formato ISO/IEC/IEEE 29119-3)	0.5	0.75	1.5	0.83	0.12
T13	MTTD, MTTR y eficacia de pruebas	0.5	0.75	1.5	0.83	0.03
T14	Eficacia de revisión, desviación y plazos	0.5	0.75	1.5	0.83	0.03
T15	Cuatro técnicas de estimación	0.75	1.25	2.5	1.38	0.08
T16	Evaluación ISO/IEC 25010 (idoneidad funcional y fiabilidad)	0.25	0.5	1.0	0.54	—
T17	AI Advisor desacoplado + respaldo por reglas	1.0	1.5	3.0	1.67	1.57
T18	Dashboard Streamlit	1.25	2.0	4.0	2.21	0.31
T19	Matriz de cumplimiento calculada (semáforo y %)	0.5	0.75	1.5	0.83	0.12
T20	Diagramas de arquitectura y pipeline	0.25	0.5	1.0	0.54	—
T21	Reporte académico en plantilla UTCJ	2.5	4.0	8.0	4.42	—
T22	Ejecución desde cero y etiqueta v1.0.0	0.25	0.5	1.0	0.54	—
T23	QA final de la matriz	0.5	1.0	2.0	1.08	—
T24	Guion de demo	0.25	0.25	0.5	0.29	—
T25	(Opcional) Encuesta de satisfacción	0.25	0.5	1.0	0.54	—

13.3 Estimación de tres puntos (PERT)

Por tarea: $E = (O + 4M + P) / 6$ y $\sigma = (P - O) / 6$. El total es la suma de las E y la desviación estándar combinada es la raíz de la suma de las varianzas (PMI, 2017).

Resultado: $E = 24.79$ h, $\sigma = 1.35$ h; con 95 % de confianza ($E \pm 2\sigma$) el proyecto tomaría entre 22.09 y 27.5 h.

13.4 Puntos de función

Conté los puntos de función sin ajustar con el método IFPUG (ISO/IEC 20926:2009): entradas externas (EI), salidas externas (EO), consultas externas (EQ), archivos lógicos internos (ILF) y archivos de interfaz externa (EIF), cada uno con un peso según su complejidad.

Tabla 47. Conteo de puntos de función de QualityOps AI (data/estimacion.json)

Tipo	Complejidad	Cantidad	Peso	PF	Funciones contadas
EI	baja	2	3	6	Ejecutar análisis (python -m qualityops --repo --salida); ejecutar quality gate (--metricas --config)
EO	media	4	5	20	metrics.json con métricas derivadas; dictamen del quality gate + resumen en Actions; informe de calidad; interpretación del AI Advisor
EO	media	5	5	25	Vistas del dashboard: resumen/cumplimiento, producto, proceso, proyecto, estimación
EQ	baja	4	3	12	Consultas por módulo: complejidad/LOC, cobertura, defectos, densidad
ILF	baja	1	7	7	reports/ (metrics.json + gate.json), mantenido por la herramienta
EIF	media	1	7	7	Historial de Git (commits, merges, tags, blame)
EIF	baja	4	5	20	Reportes de pytest/coverage; data/time_log.csv; pyproject.toml (umbrales y calendario); data/estimacion.json
Total				97	Puntos de función sin ajustar

La tasa de productividad no la tomé de la industria, porque no existe una publicada para desarrollo asistido por IA. La derivé de mi proyecto de referencia: 4.5 h / 103 PF = 0.0437 h/PF. **Resultado:** 97 PF $\times$ 0.0437 h/PF = **4.24 h**.

13.5 Estimación análoga

La estimación análoga compara con un proyecto parecido ya terminado: la herramienta [metricas](#), que desarrollé con IA en septiembre para medir el repositorio docker/secrets-engine (4.5 h reales, 103 PF). Escalé por tamaño (97/103) y apliqué un factor de ajuste de 1.13: 0.75 porque el entorno es más simple (Python puro frente a Go con módulos) y 1.5 porque el proceso es más pesado (PR, CI, evidencias y registro por tarea).

Resultado: $4.5 \times (97/103) \times 1.13 = 4.79$ h.

13.6 Comparación con las horas reales

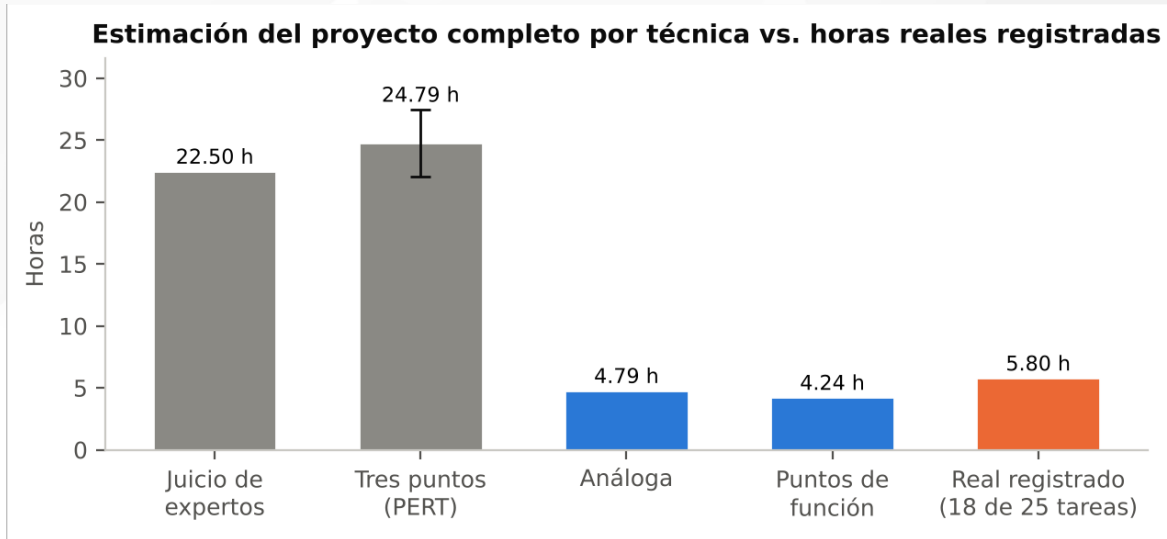


Figura 47. Estimación por técnica frente a las horas reales registradas

Qué muestra. Las cuatro estimaciones del proyecto completo y las horas reales registradas hasta ahora (naranja), con el número de tareas terminadas de la línea base. La barra de PERT incluye su rango del 95 %.

Cómo leerla. Juicio y PERT (gris) suponían programar a mano; análoga y PF (azul) usan mi productividad real con IA y quedaron cerca de lo registrado. Las tareas que faltan son de documentación (diagramas, reporte, QA final, demo), por lo que el total real seguirá creciendo hacia las estimaciones por analogía y PF.

Cómo se generó. `graficas.py` (función `estimaciones_vs_real`) con `metrics.json` y `time_log.csv`.

Tabla 48. Comparación de las técnicas

Técnica	Horas	Base	Fortaleza	Limitación
Juicio de expertos	22.5	Experiencia del autor por tarea	Rápida y por tarea	No consideró la aceleración por IA
Tres puntos (PERT)	24.79 ± 1.35	O, M, P por tarea	Expresa incertidumbre	Hereda el sesgo del juicio
Puntos de función	4.24	Tamaño funcional IFPUG	Independiente del lenguaje	Depende de la tasa h/PF
Análoga	4.79	Proyecto propio similar	Usa datos reales propios	Una sola referencia

14. PRUEBAS DE FUNCIONAMIENTO Y SALIDAS POSIBLES

En resumen: Este capítulo es una guía para poner a prueba el programa frente al profesor: qué comando ejecutar, qué salida esperar en cada escenario (aprobado, bloqueado, IA usada, IA fallida, IA no configurada) y qué evidencia del reporte muestra esa misma salida obtenida antes.

14.1 Escenarios y salidas esperadas

Tabla 49. Escenarios de prueba

Escenario	Cómo provocarlo	Salida esperada	Evidencia
Gate aprobado	<code>python -m qualityops.quality_gate</code> con los umbrales reales	3 criterios [OK], «Dictamen: APROBADO», código 0	E08a, E10h
Gate bloqueado (local)	<code>--config</code> con un TOML de cobertura mínima 95	[FALLA] en cobertura, «BLOQUEADO», código 1	E08a
Gate bloqueado (CI)	PR con una función de CC 11	Check rojo, merge deshabilitado, dictamen NO LIBERAR	E17a–d
Informe con LLM	Cargar <code>GEMINI_API_KEY</code> y ejecutar <code>qualityops.report</code>	interpretación: llm; fila «Intento de LLM: usado»	E10h, E10k
Informe sin llave	Sin <code>GEMINI_API_KEY</code>	interpretación: reglas; «no configurado (sin llave de API)»	E10g
LLM con falla	Modelo inexistente o servicio caído	fuelle: reglas y <code>error_llm</code> con el detalle	E10l, E10m
Liberación con condiciones	Gate aprobado y alguna métrica «sin datos»	LIBERAR CON CONDICIONES	E10k
Minería de defectos	<code>python -m qualityops.defects --repo .</code>	Lista de defectos con severidad, fase, MTTD, MTTR e inductor	E11f
Pruebas del programa	<code>python -m pytest -q</code>	56 passed	E10l
Otro repositorio	<code>python -m qualityops --repo C:\ruta\otro --salida reports_otro</code>	metrics.json del otro proyecto	Pruebas con repos sintéticos

14.2 Guion de demostración

1. Mostrar el repositorio y el tablero Kanban (trabajo individual y sistemático).
2. Abrir el último run de Actions: job verde, resumen con el gate y el informe de 12 secciones con «Intento de LLM: usado».
3. Mostrar el PR de demostración bloqueado por complejidad 11 (enfoque preventivo).
4. En la terminal: `python -m qualityops --repo . --salida reports` y luego el gate con el umbral de demostración (BLOQUEADO) y con el real (APROBADO).
5. Ejecutar `python -m qualityops.defects --repo .` y explicar un defecto con sus trailers (`git log -1 <hash>`).
6. Ejecutar `python -m qualityops.report` y abrir la sección 10 del informe.
7. Abrir el dashboard con `streamlit run app.py` y recorrer las 6 pestañas.

Comandos de la demostración de bloqueo local (PowerShell)

```
# Gate con umbral de demostración (debe BLOQUEAR) y con el real (debe APROBAR)
Set-Content "$env:TEMP\umbral_demo.toml" "[tool.qualityops.umbrales]`ncobertura_minima = 95.0"
python -m qualityops.quality_gate --config "$env:TEMP\umbral_demo.toml"; echo "codigo de salida: $LASTEXITCODE"
python -m qualityops.quality_gate; echo "codigo de salida: $LASTEXITCODE"
```

15. RESUMEN DE RESULTADOS Y AMENAZAS A LA VALIDEZ

En resumen: Todas las métricas solicitadas se calcularon con datos reales del proyecto y pasan sus umbrales; los límites principales son el tamaño pequeño de la muestra de defectos ($n = 3$, sin críticos) y que el tiempo real no incluye el tiempo en que la IA generó el código.

15.1 Resumen de resultados

Tabla 50. Resumen de todas las métricas (commit 9d93529)

Grupo	Métrica	Valor	Referencia / umbral
Producto	Complejidad ciclomática	Promedio 3.72, máximo 9	≤ 10 (McCabe)
Producto	Cobertura de código	88.73 %	≥ 75 % (Google)
Producto	Densidad de defectos	2.81 def/KLOC	1–25 (McConnell)
Proceso	MTTD (mediana)	0.23 h ($n = 3$)	Informativa
Proceso	MTTR (mediana)	0.05 h ($n = 3$)	< 1 h élite (DORA)
Proceso	Eficacia de las pruebas	Críticos: sin datos; todas: 100.0 %	≈ 85 % (Jones)
Proyecto	Eficacia de la revisión	66.67 %	Informativa
Proyecto	Desviación de tiempo y esfuerzo	-61.97 %	Informativa
Proyecto	Cumplimiento de plazos	33.33 %	Informativa
Estimación	Juicio / PERT / PF / análoga	22.5 / 24.79 / 4.24 / 4.79 h	—
Calidad	Grado de cumplimiento	39/44 (88.6 %)	Meta: 100 % a la entrega
Calidad	Quality gate	APROBADO	3/3 criterios

15.2 Amenazas a la validez

- **Muestra pequeña de defectos.** Con $n = 3$ los promedios de MTTD y MTTR son indicativos, no estadísticamente robustos. No hay defectos críticos, por lo que la eficacia sobre críticos no se puede calcular.
- **Tiempo real sin la generación de código por IA.** El reloj de cada tarea empezaba cuando el código generado ya existía; mide integración, verificación y PR. Esto explica gran parte de la desviación negativa.
- **Inductor de SZZ aproximado.** El defecto 3 se atribuye a c8b2162 y no a cf916cb, porque su corrección también modificó líneas de `app.py` escritas en c8b2162 y SZZ elige el commit más reciente que tocó esas líneas.
- **MTTD hasta la corrección.** No existe una marca de tiempo separada para la detección; se usa la del commit `fix:`.
- **Repartos en bloques conjuntos.** Algunas tareas se hicieron juntas (T01+T03, T09+T10, T12+T17, T13+T14, T18+T19) y su tiempo se repartió en proporción a su estimación (D11, D15).

- **Autoanálisis.** El proyecto se mide a sí mismo; aunque la opción `--repo` permite analizar otros repositorios, los valores de este reporte describen un único proyecto pequeño y de un solo autor.
- **Capturas de distintos momentos.** Cada captura indica su commit y fecha; algunos valores intermedios difieren de los finales porque el proyecto siguió creciendo. Los valores finales son los de las tablas generadas desde `metrics.json`.

III. CONCLUSIÓN

Desarrollé QualityOps AI como una pieza de infraestructura DevOps y no como una aplicación de negocio: su trabajo es decidir, en cada Pull Request, si un cambio tiene la calidad suficiente para integrarse. Los resultados muestran que la herramienta cumple ese papel con datos reales: 56 pruebas en verde, cobertura de 88.73 %, ninguna función por encima de complejidad 10, tres defectos de producto detectados y corregidos antes de producción con un tiempo mediano de restauración de 0.05 h, y un quality gate que la plataforma hace obligatorio.

Lo más valioso del proyecto no fue un número, sino la forma de obtenerlo. Al sacar los defectos del historial de Git y los tiempos de un registro hecho antes del código, cualquier cifra se puede reproducir con un comando. Esa misma disciplina expuso los errores: los de mi código (cuatro defectos reales, uno de ellos en las pruebas) y los de la IA (un proveedor retirado y una interpretación con un dato confundido). En ambos casos la verificación automática y la regla de no aceptar una salida sin ejecutarla evitaron que el error llegara al resultado final.

La IA fue útil en dos niveles: como asistente que aceleró el desarrollo (las estimaciones por analogía y puntos de función, basadas en esa productividad, fueron las más cercanas a la realidad) y como intérprete de métricas para personas no técnicas. En ambos niveles necesitó supervisión; por eso el diseño final separa el cálculo determinista de la interpretación generativa y mantiene siempre una ruta por reglas.

Los límites están claros: una muestra de defectos pequeña, sin defectos críticos, y un tiempo real que no incluye la generación de código. Como siguientes pasos propongo aplicar la herramienta a un segundo repositorio con más historia (la opción `--repo` ya lo permite), registrar por separado la hora de detección de cada defecto para medir el MTDD con más precisión, y publicar la herramienta como una *GitHub Action* reutilizable para que otros equipos la integren con una sola línea en su workflow.

IV. REFERENCIAS

- Arguelles, C., Ivanković, M., & Bender, A. (2020, 7 de agosto). *Code coverage best practices*. Google Testing Blog. <https://testing.googleblog.com/2020/08/code-coverage-best-practices.html>
- Boehm, B. W. (1981). *Software engineering economics*. Prentice-Hall.
- Conventional Commits. (s.f.). *Conventional Commits 1.0.0*. <https://www.conventionalcommits.org/en/v1.0.0/>
- Forsgren, N., Humble, J., & Kim, G. (2018). *Accelerate: The science of lean software and DevOps*. IT Revolution Press.
- GitHub. (s.f.). *GitHub Actions documentation*. <https://docs.github.com/actions>
- Google. (s.f.). *Gemini API documentation*. <https://ai.google.dev/gemini-api/docs>
- IEEE. (2014). *IEEE Std 730-2014: IEEE standard for software quality assurance processes*. IEEE.
- International Organization for Standardization & International Electrotechnical Commission. (2009). *ISO/IEC 20926:2009 Software and systems engineering — Software measurement — IFPUG functional size measurement method 2009*. ISO.
- International Organization for Standardization & International Electrotechnical Commission. (2016). *ISO/IEC 25023:2016 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Measurement of system and software product quality*. ISO.
- International Organization for Standardization & International Electrotechnical Commission. (2023). *ISO/IEC 25010:2023 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model*. ISO.
- International Organization for Standardization, International Electrotechnical Commission, & IEEE. (2017). *ISO/IEC/IEEE 12207:2017 Systems and software engineering — Software life cycle processes*. ISO.
- International Organization for Standardization, International Electrotechnical Commission, & IEEE. (2018). *ISO/IEC/IEEE 90003:2018 Software engineering — Guidelines for the application of ISO 9001:2015 to computer software*. ISO.

- International Organization for Standardization, International Electrotechnical Commission, & IEEE. (2021). *ISO/IEC/IEEE 29119-3:2021 Software and systems engineering — Software testing — Part 3: Test documentation*. ISO.
- Jones, C. (2008). *Applied software measurement: Global analysis of productivity and quality* (3.^a ed.). McGraw-Hill.
- McCabe, T. J. (1976). A complexity measure. *IEEE Transactions on Software Engineering*, SE-2(4), 308–320.
- McConnell, S. (2004). *Code complete* (2.^a ed.). Microsoft Press.
- Project Management Institute. (2017). *A guide to the project management body of knowledge (PMBOK guide)* (6.^a ed.). PMI.
- Śliwerski, J., Zimmermann, T., & Zeller, A. (2005). When do changes induce fixes? *Proceedings of the 2005 International Workshop on Mining Software Repositories*, 1–5.
- Valle Cortes, I. (2026). *QualityOps AI* [Código fuente]. GitHub.
<https://github.com/TaikoVC/qualityops-ai>